



UNIVERSITÀ DEGLI STUDI DI FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

ChargeNet

*Piattaforma per la gestione di stazioni di ricarica per veicoli
elettrici*

Repository GitHub

Candidati:

Tommaso Sottili 7138541

Luca Moracci 7140812

Marco Lampronti 7147400

Corso:

Ingegneria del software

Docente:

Prof. Enrico Vicario

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Descrizione	3
1.2	Stack Tecnologico	3
1.3	Semplificazioni adottate	4
2	Analisi dei Requisiti e Casi d'uso	5
2.1	Use Case Diagram	5
2.2	Use Case Templates	6
2.3	Mockups	13
2.3.1	Autenticazione	14
2.3.2	Area Driver	15
2.3.3	Area Station Operator	19
2.3.4	Area Energy Manager	22
2.4	Page Navigation Diagram	25
2.5	Package Diagram	25
2.6	Class Diagram	26
2.6.1	Domain Model	27
2.6.2	Business Layer	27
2.6.3	DAO	28
2.6.4	Presentation Layer	28
2.7	Sequence Diagram	29
2.7.1	Ciclo di vita del GridMonitor	30
2.7.2	Approvazione di una colonnina	31
2.7.3	Gestione dell'evento di temperatura troppo alta (Thermal Alert)	32
2.8	Database	33
2.8.1	Schema ER	34
2.8.2	Modello Relazionale	35
3	Implementazione	38
3.1	Domain Model	38
3.1.1	Users	39
3.1.2	Infrastructures	40
3.1.3	Sessions	42
3.1.4	Financials	43
3.1.5	Feedback	44
3.2	Business Layer	45
3.2.1	GridCluster	45
3.2.2	GridMonitor	46

3.2.3	LoadBalancerService	47
3.2.4	SessionService	50
3.2.5	StationService	51
3.2.6	Sicurezza e Validazione	53
3.2.7	Gestione Utente e Portafoglio	54
3.2.8	Gestione Feedback e Manutenzione	57
3.3	Controllers	57
3.4	Persistenza: il livello DAO e il mapping oggetti/tabelle	60
3.5	Idiomi e Design Pattern	63
3.5.1	Observer Pattern	63
3.5.2	Strategy Pattern	63
3.5.3	Factory Method	64
3.5.4	Singleton	65
3.5.5	DAO (Data Access Object)	66
3.5.6	Unit of Work	67
3.5.7	Service Locator	67
4	Testing	68
4.1	Strategia di Verifica	68
4.1.1	Tecnologie e Strumenti	68
4.2	Livello 1: Business Logic	69
4.2.1	SessionServiceTest	69
4.2.2	StationServiceTest	70
4.2.3	GridMonitorTest	71
4.2.4	LoadBalancerServiceTest	72
4.2.5	ChargingStrategiesTest	73
4.3	Livello 2: Persistence Layer (DAO)	74
4.3.1	PostgresStationDaoTest	74
4.3.2	PostgresSessionDaoTest	75

Capitolo 1

Introduzione

1.1 Descrizione

Il progetto **ChargeNet** è un sistema software pensato per la gestione di una rete di stazioni di ricarica per veicoli elettrici. L'obiettivo è coordinare l'intera sessione di ricarica: dalla ricerca di una colonnina libera, alla gestione del pagamento, fino alla tutela dell'infrastruttura elettrica da eventuali danni.

La piattaforma è pensata per tre figure, ciascuna con le proprie funzionalità:

- **Driver:** È l'utente finale. Utilizza il sistema per cercare le stazioni compatibili con la sua auto, far partire e monitorare la ricarica, e gestire i pagamenti tramite un portafoglio virtuale interno.
- **Station Operator:** È il proprietario delle colonnine. Registra nella piattaforma le nuove stazioni con i loro parametri tecnici (potenza, connettore, quartiere) e la tariffa che intende applicare, e ne segue l'andamento economico: guadagni maturati, energia venduta e numero di sessioni erogate. Non può decidere però da solo se una colonnina entra in rete, infatti ogni nuova registrazione resta in attesa dell'approvazione dell'Energy Manager.
- **Energy Manager:** È il supervisore della rete e ha un doppio ruolo. Da un lato autorizza, vale a dire approva o rifiuta le colonnine proposte dagli operatori e, approvandole, ne fissa la quota di tariffa trattenuta dalla piattaforma; interviene inoltre sulle colonnine segnalate per qualità scadente, sospendendole o archiviando la segnalazione. Dall'altro ha il compito di supervisionare. Ha infatti una dashboard che gli mostra in tempo reale carico e temperatura di ogni trasformatore. La protezione dell'infrastruttura in caso di surriscaldamento non è però una sua azione manuale: è il sistema stesso a reagire in autonomia interrompendo le ricariche sul ramo in sovraccarico (*Load Balancing*, descritto nel paragrafo 3.5.1).

1.2 Stack Tecnologico

- **Linguaggio:** Java, essendo un linguaggio fortemente orientato agli oggetti, ci ha permesso di modellare in modo chiaro le entità del dominio e le loro interazioni.
- **Interfaccia grafica:** JavaFX con FXML, che separa la descrizione del layout (i file `.fxml`) dalla logica di schermata (i **Controller**).

- **Persistenza:** database relazionale PostgreSQL. In sviluppo si è usato H2 in modalità `MODE=PostgreSQL`, così da accettare la stessa sintassi SQL e rendere veloce e semplice il passaggio a PostgreSQL.
- **Accesso ai dati:** JDBC con query scritte a mano, senza ORM, per controllo esplicito della traduzione oggetti/tabelle.
- **Sicurezza:** password protette con hash BCrypt.
- **Build e dipendenze:** Maven.
- **Test:** JUnit 5 con Mockito per l'isolamento tramite mock.
- **Intelligenza Artificiale:** Per l'interfaccia grafica, il refactoring e l'ottimizzazione del codice abbiamo usufruito di Intelligenza Artificiale Generativa, nello specifico dei modelli di Gemini e Claude.

1.3 Semplificazioni adottate

Trattandosi di un progetto universitario, con tempo e risorse limitati, sono state adottate alcune semplificazioni, cercando comunque di restare vicini a un sistema realmente utilizzabile. Le scelte principali sono le seguenti:

- La geolocalizzazione non deriva da un indirizzo reale ma dal quartiere scelto (per le colonnine) o da una lettura GPS simulata (per il Driver); in produzione si userebbe un servizio di geocoding.
- La distanza tra due punti è calcolata con formula euclidea, più che sufficiente all'unico scopo di ordinare le colonnine per vicinanza.
- Non esiste un sistema di pagamento verso l'esterno: ricariche del portafoglio e guadagni sono movimenti interni.
- I coefficienti di calore dei trasformatori sono tarati per rendere il surriscaldamento osservabile in un tempo breve. Nella realtà un trasformatore si surriscalda per l'effetto cumulativo di molte ricariche contemporanee sullo stesso ramo; nella nostra applicazione, pensata per un solo utente alla volta, non è possibile simulare più sessioni in parallelo per raggiungere lo stesso effetto. Per questo motivo il calore generato da una singola sessione è stato aumentato, così da poter osservare l'intero meccanismo (accumulo, soglia, notifica, reazione) anche con una sola ricarica attiva, senza alterarne la logica.
- Il Driver una volta che ha un piano di abbonamento non può passare ad un altro di prezzo inferiore, non abbiamo implementato la scadenza temporale perché non strettamente necessaria al funzionamento della demo.

Capitolo 2

Analisi dei Requisiti e Casi d'uso

Questo capitolo descrive il processo di progettazione del sistema seguendo un approccio che va dall'esterno verso l'interno, partendo da come gli utenti interagiscono con l'applicazione, definendo i requisiti e l'interfaccia grafica, per poi scendere nei dettagli tecnici dell'architettura software e della struttura del database.

2.1 Use Case Diagram

Il diagramma dei casi d'uso offre una visione d'insieme delle funzionalità del sistema, mostrando quali azioni possono compiere i tre attori principali (Driver, Station Operator ed Energy Manager) all'interno della piattaforma.

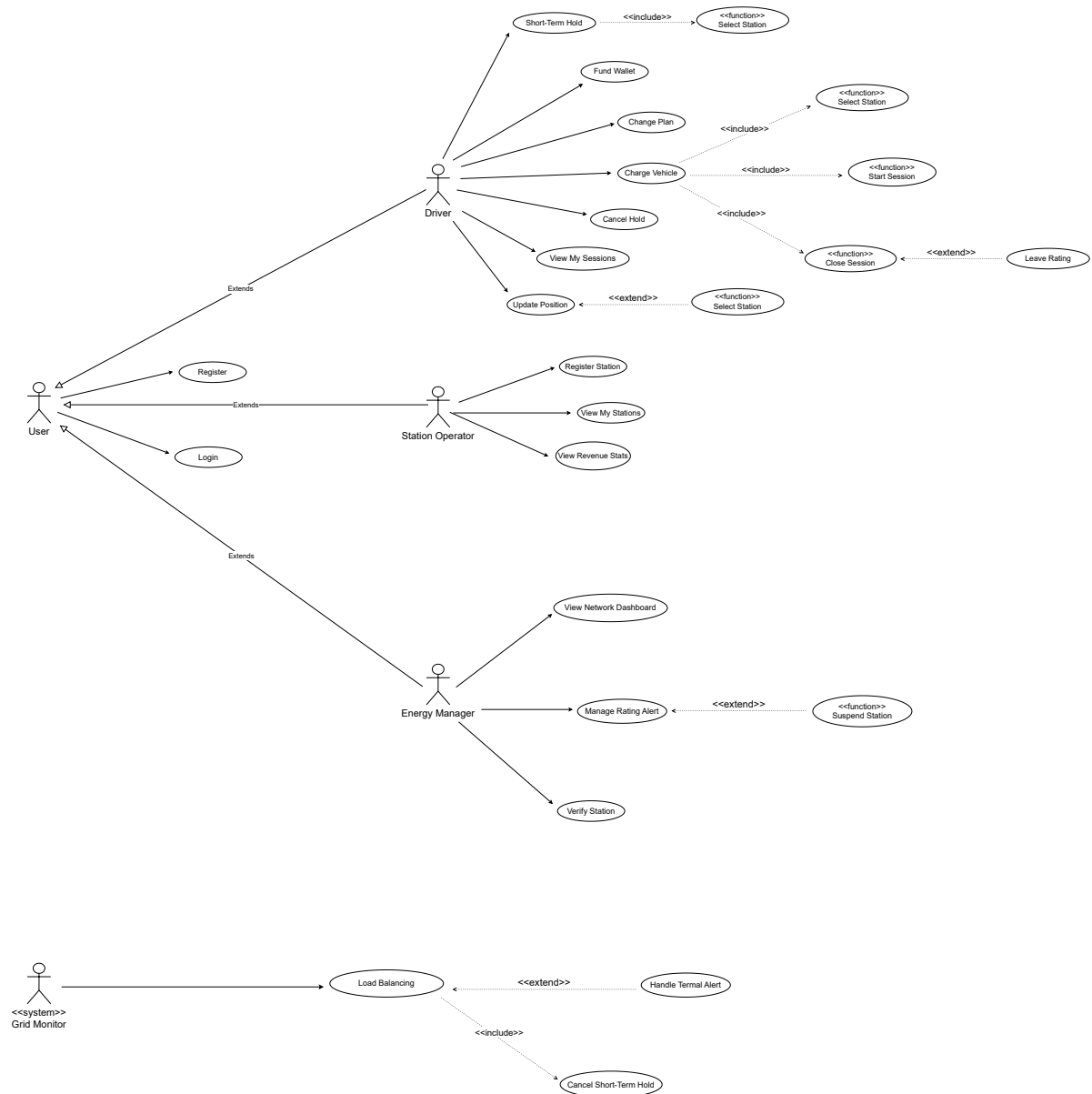


Figura 2.1: Diagramma dei casi d'uso

2.2 Use Case Templates

Per evitare ambiguità, i casi d'uso più importanti sono stati documentati con dei template strutturati, che descrivono passo dopo passo il flusso degli eventi, le precondizioni necessarie e i possibili scenari di errore.

Tabella 2.1: Caso d'Uso: Ricarica Veicolo

Campo	Contenuto
Nome	Ricarica Veicolo
Livello	Obiettivo Utente
Descrizione	Il Conducente avvia una sessione di ricarica su una colonnina selezionata, scegliendo la strategia Fast Charge oppure Eco Charge.
Pre-condizioni	Il Conducente ha effettuato l'accesso. Il saldo del portafoglio copre almeno il costo di 5 kWh presso la colonnina scelta.
Post-condizioni	La sessione è in stato COMPLETED , il costo finale è stato scalato dal portafoglio, è registrata una Transaction di tipo CHARGE e la colonnina è tornata ACTIVE .
Attori	Conducente, Grid Monitor (Sistema)
Unit Test	<code>SessionServiceTest</code>
Flusso Principale	1. Il Conducente seleziona una colonnina dall'elenco di quelle disponibili. 2. Conferma la scelta di una colonnina ACTIVE, oppure RESERVED da lui stesso. 3. Sceglie la strategia di ricarica: Fast Charge oppure Eco Charge. 4. Inserisce la percentuale di batteria attuale del veicolo. 5. Il sistema verifica la compatibilità del connettore, la disponibilità della colonnina e che il saldo copra almeno 5 kWh. 6. Il sistema porta la colonnina in stato BUSY e apre la sessione. 7. Il GridMonitor fa avanzare periodicamente la batteria e addebita il costo di ogni tick (Tab. 2.4). 8. Il Conducente interrompe la ricarica. 9. Il sistema calcola i kWh totali erogati e il costo finale, emette la ricevuta e accredita allo Station Operator la sua quota. 10. Il sistema porta la sessione a COMPLETED e riporta la colonnina ad ACTIVE.

Continua nella pagina successiva...

Tabella 2.1 – Continua dalla pagina precedente

Campo	Contenuto
Flussi Alternativi	1. Colonnina non disponibile: 1.1. Le colonnine BUSY, OVERLOADED, SUSPENDED, incompatibili col connettore o RESERVED da altri non compaiono in elenco: il Conducente non può selezionarle. 5. Credito insufficiente: 5.1. Se il saldo non copre il costo di 5 kWh, il sistema rifiuta l'apertura e mostra il messaggio <i>"Credito insufficiente per avviare la sessione"</i>. 5.2. Il controllo appartiene all'entità di dominio ChargingSession, quindi il rifiuto avviene prima di qualsiasi accesso al database. 8. Batteria al 100%: 8.1. Se la batteria raggiunge il 100%, la sessione si chiude da sola senza attendere il comando del Conducente, seguendo poi i passi 9 e 10. 7. Allarme Termico: 7.1. Se la temperatura del trasformatore supera i 90 °C, il LoadBalancerService porta tutte le colonnine di quel trasformatore a OVERLOADED e ne interrompe le sessioni in corso. 7.2. La sessione passa a INTERRUPTED_THERMAL e la UI mostra il messaggio <i>"Arresto di emergenza: surriscaldamento del sistema"</i>. 7.3. Il Conducente viene rimborsato del costo dell'ultimo tick, cioè dell'energia pagata ma non erogata al momento dello stacco.

Tabella 2.2: Caso d'Uso: Prenotazione a Breve Termine

Campo	Contenuto
Nome	Prenotazione a Breve Termine
Livello	Obiettivo Utente
Descrizione	Il Conducente riserva per 15 minuti una colonnina ACTIVE, per assicurarsene la disponibilità al proprio arrivo.
Pre-condizioni	Il Conducente ha effettuato l'accesso. Non ha già un'altra prenotazione attiva. La colonnina scelta è ACTIVE ed è compatibile con il connettore del veicolo.

Continua nella pagina successiva...

Tabella 2.2 – Continua dalla pagina precedente

Campo	Contenuto
Post-condizioni	La colonnina è in stato RESERVED , il campo reserved_by_id punta al Conducente e expiration_timestamp è impostato a 15 minuti dall'istante corrente. La colonnina resta visibile al solo Conducente che l'ha prenotata.
Attori	Conducente
Unit Test	StationServiceTest
Flusso Principale	1. Il Conducente seleziona una colonnina ACTIVE dall'elenco di quelle disponibili. 2. Conferma di volerla prenotare. 3. Il sistema tenta l'acquisizione con un UPDATE condizionale, che modifica la riga solo se la colonnina risulta ancora ACTIVE. 4. Il sistema porta la colonnina a RESERVED e ne registra il Conducente come titolare. 5. Il sistema imposta expiration_timestamp a 15 minuti dall'istante corrente. 6. Il sistema conferma la prenotazione mostrando l'orario esatto di scadenza.
Flussi Alternativi	3. Colonnina non più disponibile: 3.1. Se nel frattempo un altro Conducente l'ha prenotata o occupata, l'UPDATE non aggiorna alcuna riga e il sistema nega la prenotazione. 3.2. La transazione viene annullata con un rollback e l'oggetto in memoria non viene modificato, così che memoria e database non divergano. 3.3. Il sistema mostra il messaggio <i>"La colonnina non è più disponibile"</i> e si torna al passo 1. Nota sulla concorrenza: • È il database stesso a impedire la doppia prenotazione: la condizione sullo stato ACTIVE è parte dell'UPDATE, quindi fra due Conducenti simultanei ne può vincere uno solo, senza alcun blocco applicativo. Nota sulla scadenza: • Se i 15 minuti trascorrono senza che venga avviata una sessione, il GridMonitor riporta la colonnina ad ACTIVE (Tab. 2.4).

Tabella 2.3: Caso d'Uso: Gestione di un Alert di Qualità

Campo	Contenuto
Nome	Gestione di un Alert di Qualità
Livello	Obiettivo Utente
Descrizione	L'Energy Manager esamina una segnalazione aperta automaticamente dal sistema su una colonnina la cui qualità percepita è degradata, e decide se sospenderla oppure archiviare la segnalazione.
Pre-condizioni	L'Energy Manager ha effettuato l'accesso. Esiste almeno un RatingAlert in stato PENDING . La segnalazione non nasce da una singola recensione negativa: è aperta dal RatingService quando la media della colonnina scende sotto 2.0 con almeno 5 valutazioni. Per una stessa colonnina non può esistere più di una segnalazione aperta.
Post-condizioni	La segnalazione passa a RESOLVED_SUSPENDED oppure a RESOLVED_IGNORED , con la nota del Manager registrata. Solo nel primo caso la colonnina passa a SUSPENDED ; archiviando, il suo stato resta invariato.
Attori	Energy Manager
Unit Test	RatingServiceTest
Flusso Principale	1. L'Energy Manager apre la schermata di gestione delle segnalazioni di qualità. 2. Il sistema mostra le sole segnalazioni in stato PENDING, con nome della colonnina, media attuale delle valutazioni, indicatore di gravità e tempo trascorso dall'apertura. 3. L'Energy Manager seleziona una segnalazione e sceglie una delle due azioni possibili: • Archivia la segnalazione: 4a. Inserisce una nota di motivazione, obbligatoria, e conferma. 5a. Il sistema porta la segnalazione a RESOLVED_IGNORED e ne registra la nota. Lo stato della colonnina non viene toccato. • Sospendi la colonnina («extend»): 4b. Inserisce la motivazione della sospensione, obbligatoria, e conferma. 5b. Il sistema porta la segnalazione a RESOLVED_SUSPENDED e la colonnina a SUSPENDED, in un'unica transazione. 4. La segnalazione scompare dalla coda di quelle aperte.

Continua nella pagina successiva...

Tabella 2.3 – Continua dalla pagina precedente

Campo	Contenuto
Flussi Alternativi	2. Segnalazione già gestita: 2.1. Se nel frattempo la segnalazione non è più in stato PENDING, ad esempio perché chiusa da un altro Manager, il sistema rifiuta l'operazione. 2.2. La decisione non spetta al servizio ma all'entità di dominio: RatingAlert applica la risoluzione solo se la segnalazione è ancora PENDING, quindi una già chiusa non può essere risolta una seconda volta. 4. Annullamento: 4.1. Se il Manager chiude il dialogo o lascia vuota la nota, l'operazione viene abbandonata e nessuna modifica viene salvata. Nota sull'atomicità: • Nella sospensione, l'aggiornamento della segnalazione e quello della colonnina appartengono a un'unica transazione: se una delle due scritture fallisce, il rollback annulla anche l'altra, evitando una colonnina sospesa con la segnalazione ancora aperta, o viceversa.

Tabella 2.4: Caso d'Uso: Aggiornamento dello Stato del Sistema

Campo	Contenuto
Nome	Aggiornamento dello Stato del Sistema
Livello	Obiettivo di Sistema
Descrizione	Il GridMonitor è un thread <i>daemon</i> avviato insieme all'applicazione, ed è l'unico componente che agisce di propria iniziativa: tutti gli altri servizi vengono invocati su richiesta. Ogni 5 secondi fa avanzare la simulazione.
Pre-condizioni	L'applicazione è avviata e il GridMonitor è in esecuzione. Il ciclo produce effetti osservabili se esiste almeno una sessione ACTIVE oppure una prenotazione RESERVED scaduta.
Post-condizioni	Le prenotazioni scadute sono tornate ACTIVE . Ogni sessione attiva ha batteria, kWh erogati e costo aggiornati, e l'importo del tick è stato scalato dal portafoglio del Conducente. Temperatura e carico di ogni trasformatore riflettono il calore generato nel tick.
Attori	Grid Monitor (Sistema)
Unit Test	GridMonitorTest

Continua nella pagina successiva...

Tabella 2.4 – Continua dalla pagina precedente

Campo	Contenuto
Flusso Principale	1. Il timer del <code>GridMonitor</code> scatta, ogni 5 secondi. 2. Il sistema libera le prenotazioni scadute («include»): le colonnine <code>RESERVED</code> il cui <code>expiration_timestamp</code> è ormai passato tornano <code>ACTIVE</code>. 3. Per ogni sessione <code>ACTIVE</code> il sistema calcola l'energia erogata nel tick, pari alla potenza della colonnina moltiplicata per la frazione d'ora corrispondente a 5 secondi. L'energia erogata non dipende dalla strategia: la batteria avanza alla stessa velocità in entrambe le modalità, perché la potenza è una caratteristica fisica della colonnina. 4. La strategia determina invece il <i>costo</i> del tick e il <i>calore</i> prodotto: • Fast Charge: il Conducente paga la tariffa piena, cioè il prezzo base della colonnina; la sessione genera il doppio del calore. • Eco Charge: sullo stesso prezzo base è applicato un ulteriore sconto del 10%; la sessione genera metà del calore. 5. Il sistema addebita il costo del tick al portafoglio del Conducente e salva sessione e saldo in un'unica transazione. 6. Il sistema somma il calore prodotto da tutte le sessioni, lo raggruppa per trasformatore e lo applica a ciascuno di essi, che ne ricalcola temperatura e percentuale di carico. 7. Il sistema esegue infine il controllo di qualità: se una colonnina ha una media inferiore a 2.0 con almeno 5 valutazioni, viene aperta una segnalazione (Tab. 2.3).

Continua nella pagina successiva...

Tabella 2.4 – Continua dalla pagina precedente

Campo	Contenuto
Flussi Alternativi	3. Batteria al 100%: 3.1. Se il tick porta la batteria al 100%, la sessione passa a COMPLETED e viene chiusa. 3.2. Il sistema emette la ricevuta con il costo accumulato tick dopo tick e riporta la colonnina ad ACTIVE. 5. Credito esaurito durante la ricarica: 5.1. Prima dell’addebito il sistema verifica che il saldo copra il costo del tick. 5.2. Se non lo copre, il tick non viene addebitato: il portafoglio non può in nessun caso scendere sotto lo zero. 5.3. La sessione viene chiusa d’autorità seguendo lo stesso percorso della conclusione normale: ricevuta emessa con il costo effettivamente accumulato e colonnina di nuovo ACTIVE. 5.4. Il Conducente potrà avviare una nuova ricarica solo dopo aver ricaricato il portafoglio. 6. Allarme Termico («extend»): 6.1. Se applicando il calore la temperatura di un trasformatore supera i 90 °C, il trasformatore notifica i propri <i>observer</i> con l’evento THERMAL_ALERT. 6.2. Il LoadBalancerService porta prima tutte le colonnine di quel trasformatore allo stato OVERLOADED, in un’unica transazione atomica, impedendo l’avvio di nuove ricariche sul ramo. 6.3. Solo dopo interrompe le sessioni ancora in corso: ciascuna passa a INTERRUPTED_THERMAL, il Conducente viene rimborsato del costo dell’ultimo tick e allo Station Operator viene comunque accreditata la sua quota sull’energia realmente erogata. Ogni chiusura forzata ha una propria transazione indipendente: il fallimento di un rimborso non annulla quelli già andati a buon fine. 6.4. Quando la temperatura ridiscende sotto i 70 °C, il trasformatore emette l’evento COOLING_COMPLETE e le colonnine rimaste OVERLOADED tornano ACTIVE. La distanza fra le due soglie evita che il sistema oscilli attorno a un unico valore.

2.3 Mockups

Di seguito sono riportate le schermate dell’applicazione, raggruppate per attore e presentate nell’ordine in cui l’utente le incontra. L’obiettivo di questa sezione è mostrare come le funzionalità descritte nei casi d’uso siano state rese accessibili in modo immediato, e come a ciascun ruolo venga presentato soltanto ciò che gli compete: le tre aree (Driver, Station Operator ed Energy Manager) sono separate fin dal login, che instrada l’utente verso la dashboard del proprio ruolo.

2.3.1 Autenticazione

Login

La schermata di accesso raccoglie email e password e delega la verifica all'`AuthService`, che confronta l'hash BCrypt della password. Riconosciuto l'utente, il sistema lo indirizza automaticamente alla dashboard corrispondente al suo ruolo.

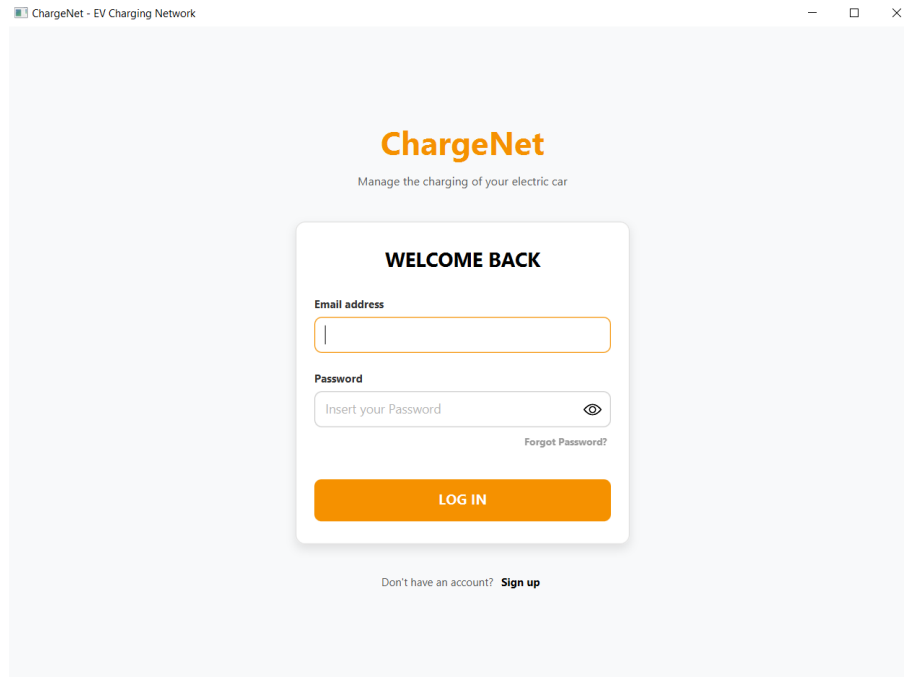


Figura 2.2: Schermata di login.

Registrazione

La registrazione raccoglie i dati del nuovo account e il ruolo desiderato. Il flusso cambia in base al ruolo scelto: solo al Driver vengono richiesti i dati del veicolo (tipo di connettore e capacità della batteria) e viene assegnata una posizione GPS iniziale simulata, necessaria per ordinare le colonnine per vicinanza.

ChargeNet - EV Charging Network

ChargeNet
Manage the charging of your electric car

CREATE ACCOUNT

Full Name

Email address

Password

Select Role

REGISTER NOW

Already have an account? [Log in](#)

Figura 2.3: Schermata di registrazione.

2.3.2 Area Driver

Menu del Driver

È il contenitore della dashboard del Driver: mostra il saldo del portafoglio e dà accesso alle quattro schermate della sua area (esplorazione delle colonnine, ricarica del portafoglio, storico e profilo). Il pulsante *Update Location* simula una nuova lettura GPS e ne persiste il risultato, così da riordinare le colonnine per vicinanza. Se una ricarica è ancora in corso, compare inoltre una card *Resume* che riporta alla sessione attiva: la ricarica infatti prosegue anche quando il Driver esce dalla schermata, perché ad avanzarla è il **GridMonitor** in background e non l'interfaccia.

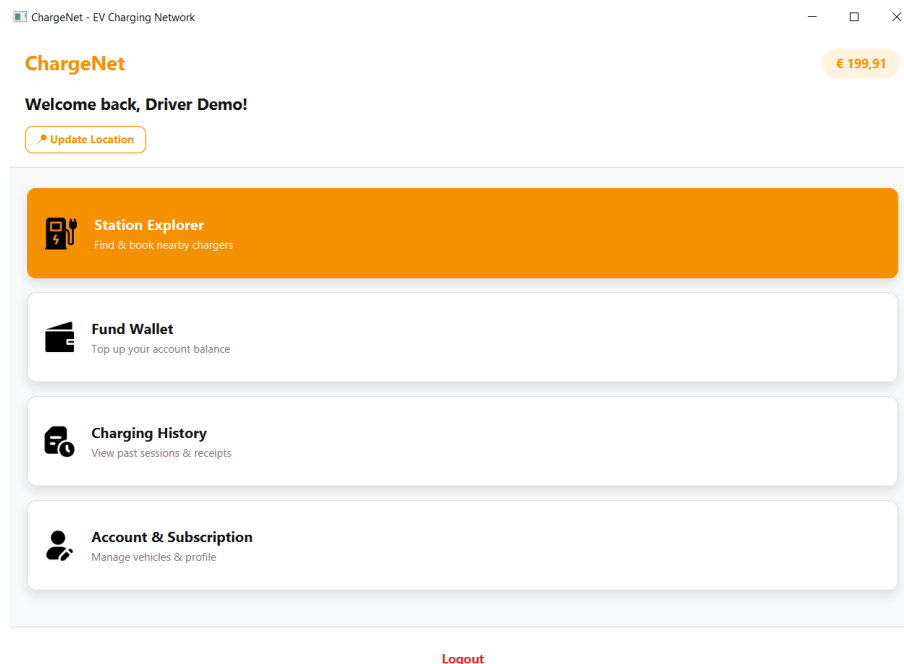


Figura 2.4: Menu del Driver.

Esplorazione delle colonnine

La schermata mostra *soltanto* le colonnine che il Driver può realmente usare: attive, compatibili con il connettore del suo veicolo e ordinate dalla più vicina alla più lontana. Le colonnine occupate, in sovraccarico o incompatibili non vengono mostrate affatto, perché proporre opzioni non selezionabili sarebbe un difetto di esperienza d'uso. Fanno eccezione le colonnine prenotate dal Driver stesso, che restano visibili con l'apposita etichetta.

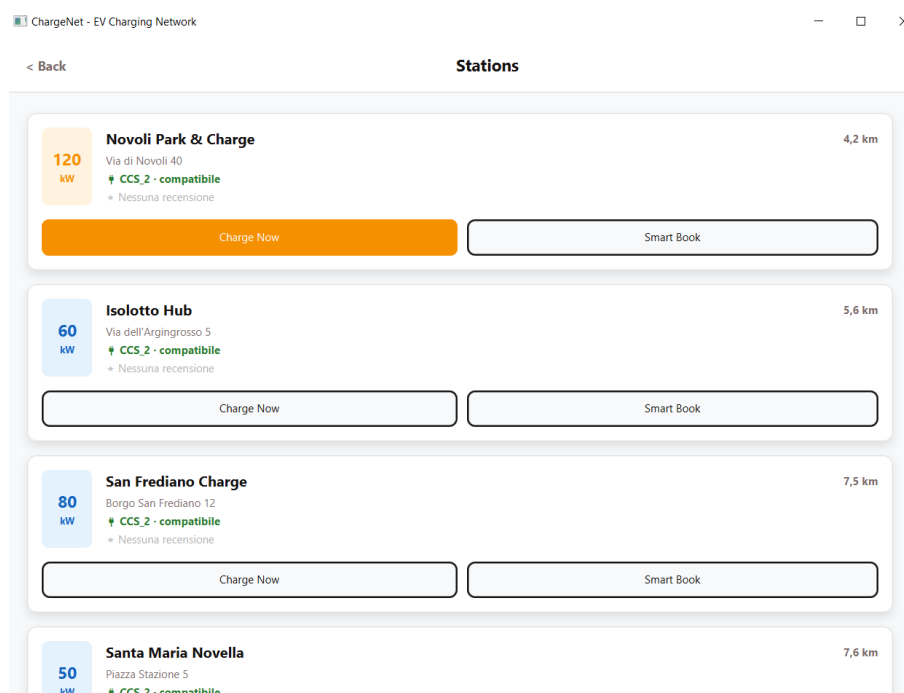


Figura 2.5: Esplorazione delle colonnine disponibili.

Sessione di ricarica in tempo reale

Il Driver sceglie la modalità (Fast o Eco), indica la percentuale di batteria iniziale e avvia la ricarica. Da quel momento la schermata non calcola nulla: rilegge lo stato della sessione dal database una volta al secondo e aggiorna a video batteria, energia erogata, costo e saldo residuo.

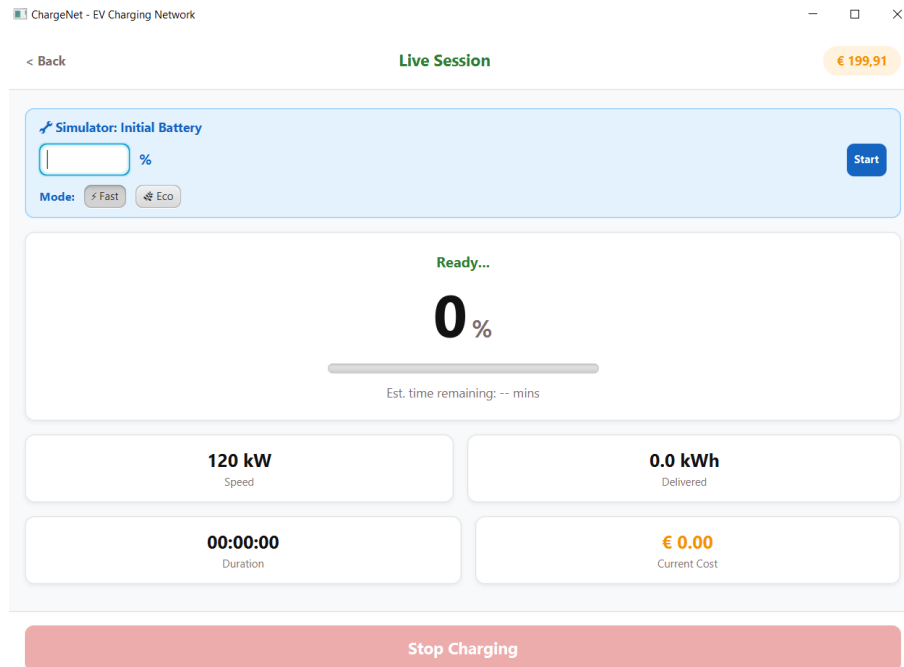


Figura 2.6: Sessione di ricarica in tempo reale.

Ricarica del portafoglio

La schermata permette di aggiungere fondi al portafoglio, scegliendo uno degli importi predefiniti oppure digitandone uno personalizzato; ogni versamento genera una transazione registrata nello storico contabile. Il metodo di pagamento mostrato in figura è solo un'aggiunta grafica, infatti come dichiarato tra le semplificazioni (paragrafo 1.3), il portafoglio è interno alla piattaforma e non esiste alcuna integrazione con circuiti di pagamento reali. La conferma non contatta alcun servizio esterno: si limita ad accreditare l'importo sul saldo del Driver e a registrarne il movimento.

ChargeNet - EV Charging Network

< Back Fund Wallet

Current Balance

€ 199,91

Select Amount

+ € 10 + € 20 + € 50

Custom: € 0.00

Payment Method

VISA **** * 4242 Expires 12/28

Total to pay: € 0.00

Confirm & Pay

Figura 2.7: Ricarica del portafoglio.

Storico delle ricariche

Elenca le sessioni concluse del Driver, ciascuna con data, energia erogata e costo, e ne riporta i totali: spesa complessiva ed energia totale prelevata. La valutazione di una ricarica non avviene qui, ma nella finestra proposta al Driver al termine della sessione.

ChargeNet - EV Charging Network

< Back Charging History

Total Spent	Total Energy
€ 96,41	213,2 kWh

Recent Sessions

Firenze Duomo Hub 04 lug • 17:53	€ 14,40 32,0 kWh
Novoli Park & Charge 02 lug • 17:53	€ 12,04 28,0 kWh
Piazzale Michelangelo 30 giu • 17:53	€ 20,00 40,0 kWh
Santa Maria Novella 27 giu • 17:53	€ 7,20 18,0 kWh
Oltrarno Fast 23 giu • 17:53	€ 5,64 12,0 kWh

Figura 2.8: Storico delle ricariche del Driver.

Profilo e abbonamento

Riporta il saldo del portafoglio e i dati del veicolo (tipo di connettore e capacità della batteria, in sola lettura), e permette di cambiare il piano di abbonamento. I tre piani sono presentati con il rispettivo canone mensile e con lo sconto che ciascuno garantisce: come mostra la figura, lo sconto si applica *esclusivamente alla quota di tariffa trattenuta dalla piattaforma* (0% per BASIC, 15% per PLUS, 30% per PREMIUM), mai alla quota dell'operatore. Il passaggio a un piano superiore addebita la sola differenza di canone rispetto al piano corrente, mentre i passaggi a un piano inferiore sono bloccati; il pulsante di conferma resta disabilitato finché il Driver non seleziona un piano diverso da quello attivo.

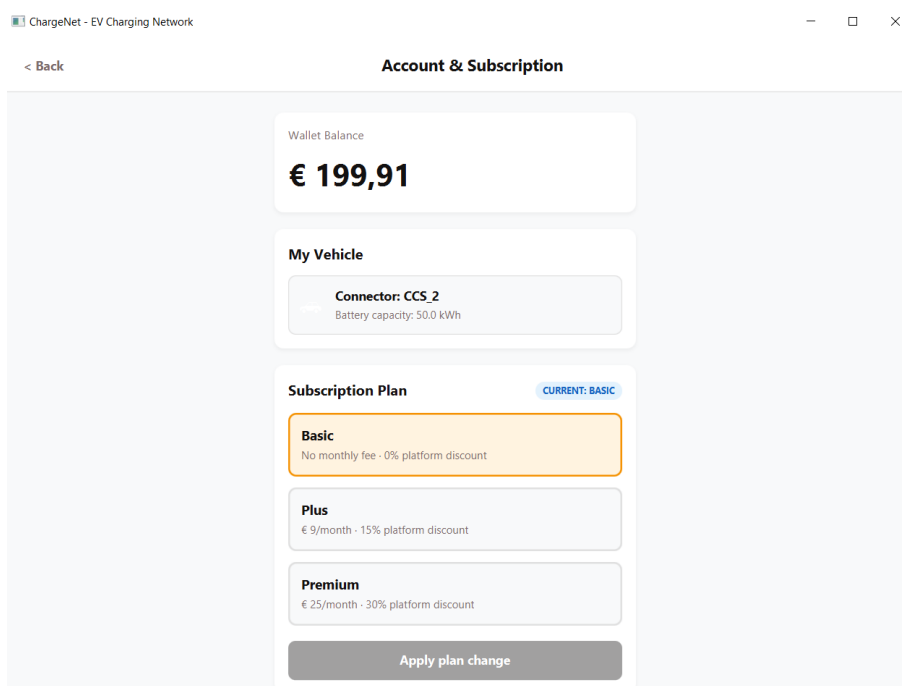


Figura 2.9: Profilo utente e gestione dell'abbonamento.

2.3.3 Area Station Operator

Menu dell'Operatore

Contenitore della dashboard dell'operatore, da cui si accede alla gestione delle proprie colonnine e ai rendiconti economici.

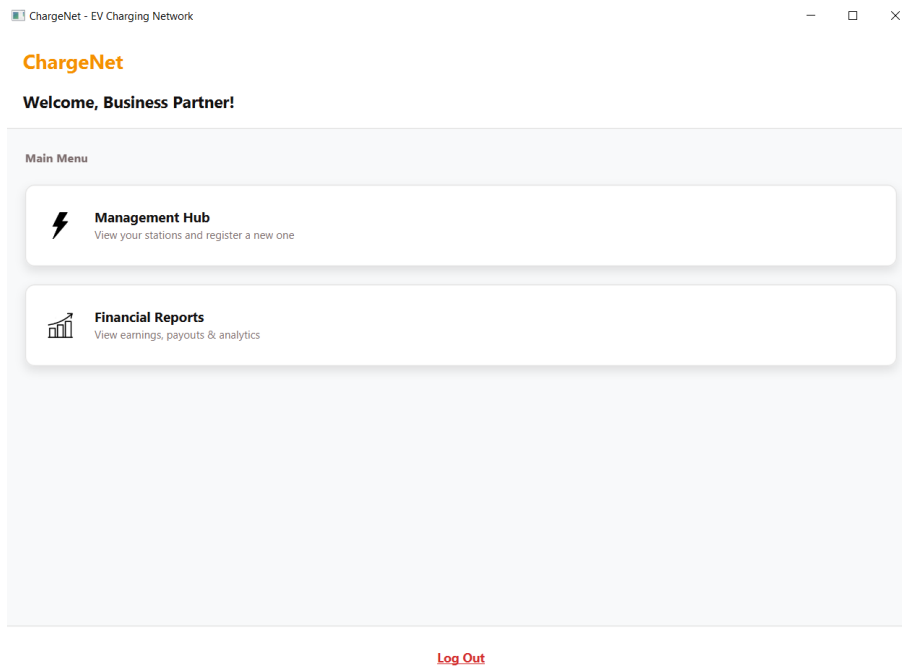


Figura 2.10: Menu dello Station Operator.

Gestione delle colonnine

Elenca le colonnine possedute dall'operatore con il loro stato operativo corrente (attiva, occupata, in sovraccarico, in attesa di approvazione, rifiutata), e dà accesso rapido alla registrazione di una nuova colonnina.

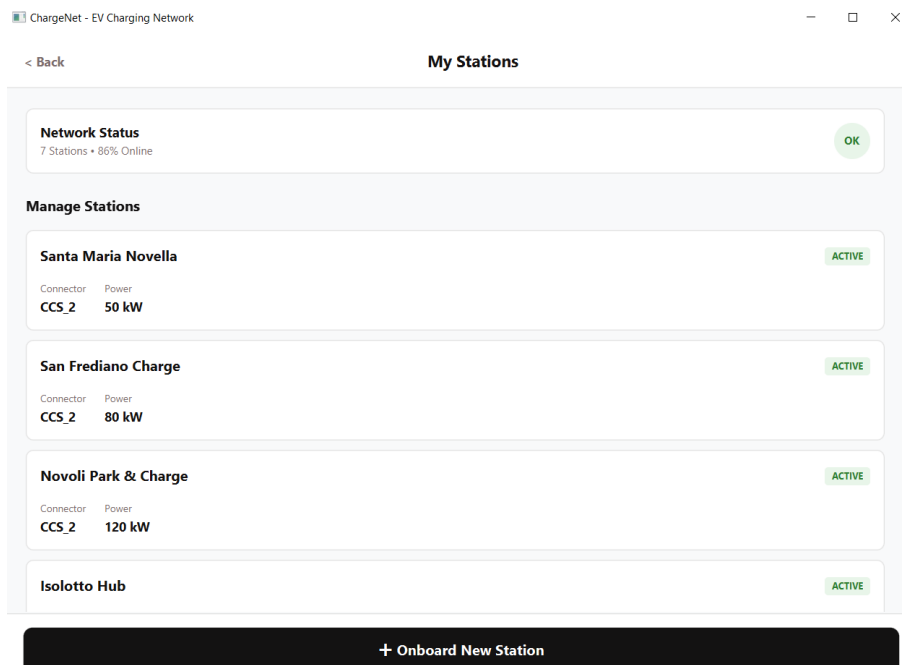


Figura 2.11: Gestione delle colonnine dell'operatore.

Registrazione di una nuova colonnina

Il modulo raccoglie i parametri della nuova colonnina: nome, indirizzo, quartiere, potenza, tipo di connettore e tariffa dell'operatore. Il sistema rifiuta i parametri fisicamente incoerenti, per esempio una potenza superiore al limite del connettore scelto. La colonnina appena registrata **non entra subito in rete**: nasce in stato `PENDING_VERIFICATION` e resta in attesa dell'approvazione dell'Energy Manager.

The screenshot shows a web application window titled "ChargeNet - EV Charging Network". Inside, there's a "New Station" form. At the top left is a "< Back" button, and at the top right is a "Clear" button. The form is divided into two main sections: "1. Location & Identity" and "2. Technical Specifications".

1. Location & Identity

- Station Name**: A text input field with a placeholder "e.g. Firenze SMN Fast".
- Full Address**: A text input field with a placeholder "e.g. Piazza della Stazione, Firenze".
- District**: A dropdown menu with the placeholder "Seleziona quartiere".

2. Technical Specifications

- Max Power (kW)**: A text input field with the value "150".
- Tariff (€/kWh)**: A text input field with the value "0.30".
- Connector Type**: A dropdown menu with the placeholder "Seleziona connettore".

At the bottom of the form is a large black button labeled "Deploy Station".

Figura 2.12: Registrazione di una nuova colonnina.

Rendiconti economici

Riepiloga i guadagni maturati dall'operatore, l'energia venduta e il numero di sessioni erogate dalle sue colonnine. I valori non sono precalcolati né scritti a mano nell'interfaccia: sono ricavati dalle stesse query che alimentano il resto del sistema, e vengono riletti dal database all'apertura della schermata, perché il guadagno cambia a ogni sessione conclusa da un Driver.

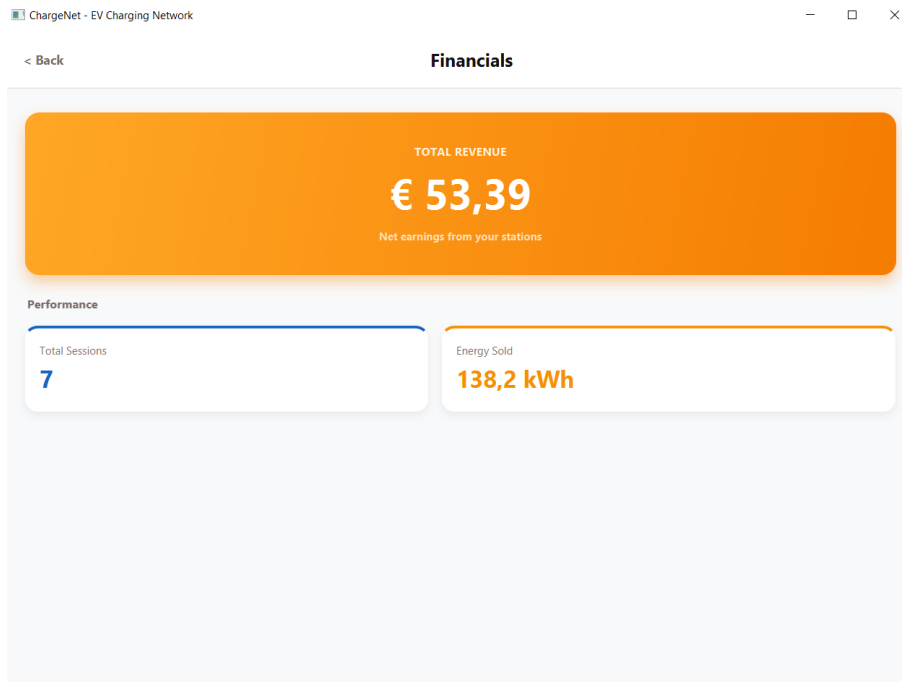


Figura 2.13: Rendiconti economici dell'operatore.

2.3.4 Area Energy Manager

Menu dell'Energy Manager

Contenitore della dashboard del Manager, da cui si accede alle tre schermate di supervisione della rete: la coda di approvazione, la gestione delle segnalazioni di qualità e la dashboard di affidabilità.

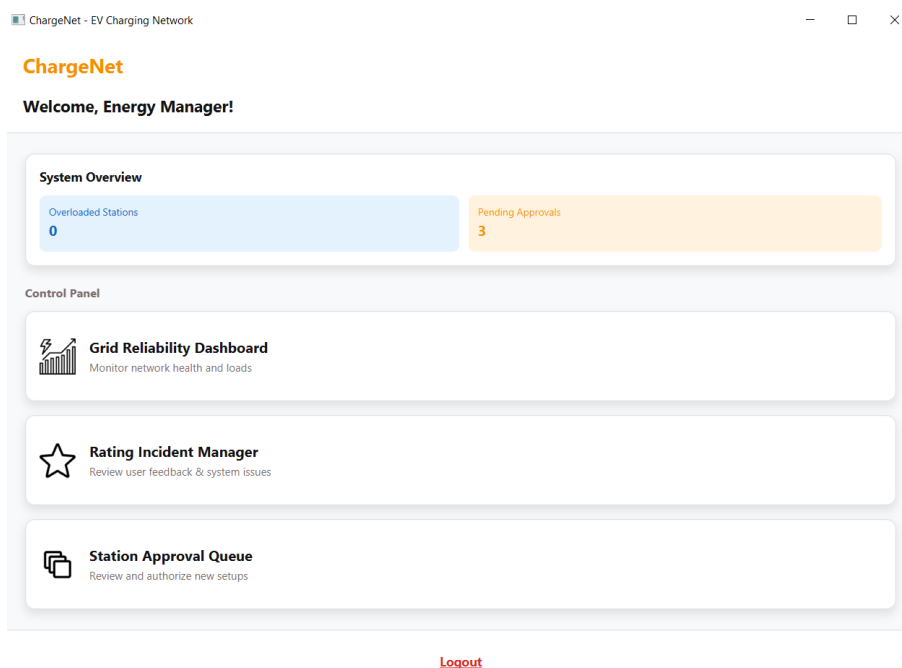


Figura 2.14: Menu dell'Energy Manager.

Coda di approvazione

Raccoglie le colonnine in attesa di verifica registrate dagli operatori. Il Manager può approvarle o rifiutarle. Approvare non è un semplice assenso: al momento dell'approvazione viene richiesta la **quota di tariffa della piattaforma** in euro per kWh, che da quel momento entra nel calcolo del costo di ogni ricarica su quella colonnina.

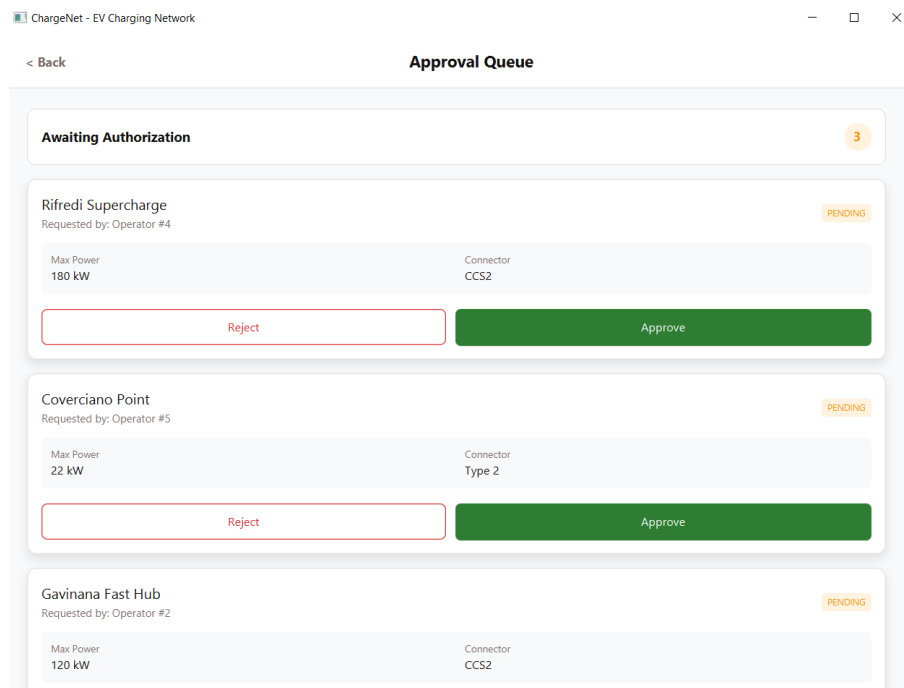


Figura 2.15: Coda di approvazione delle nuove colonnine.

Segnalazioni di qualità

Elenca gli alert generati automaticamente dal sistema quando una colonnina accumula valutazioni troppo basse. Il Manager può sospendere la colonnina oppure archiviare la segnalazione come falso allarme, motivando in entrambi i casi la decisione (si veda il caso d'uso in Tab. 2.3).

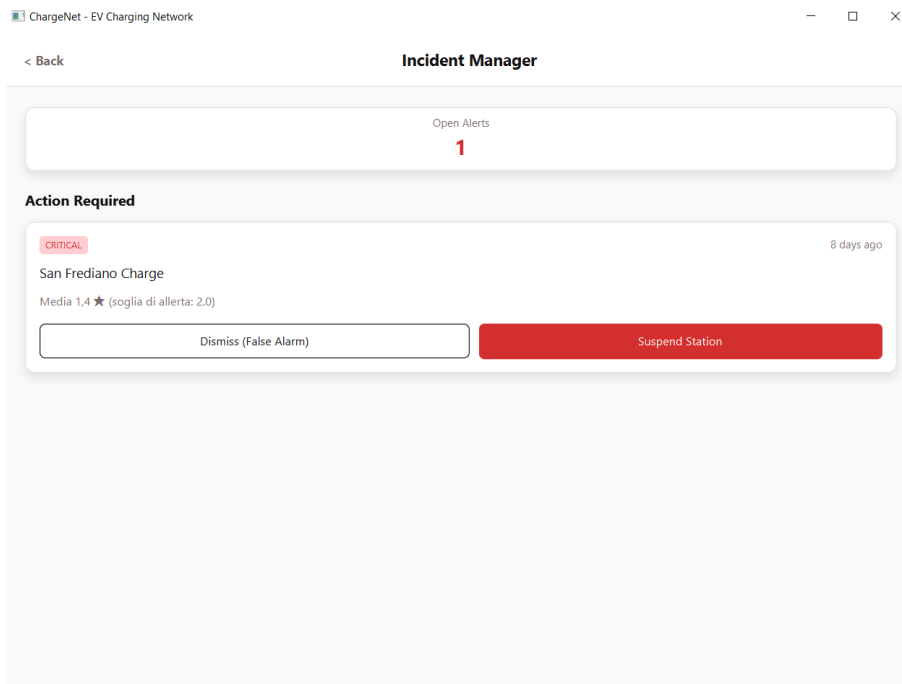


Figura 2.16: Gestione delle segnalazioni di qualità.

Dashboard di affidabilità della rete

Mostra, per ciascun trasformatore, il carico e la temperatura correnti, con evidenza visiva di un eventuale allarme termico. È una schermata di sola osservazione: la protezione della rete in caso di surriscaldamento non è un'azione manuale del Manager, ma una reazione automatica del sistema.

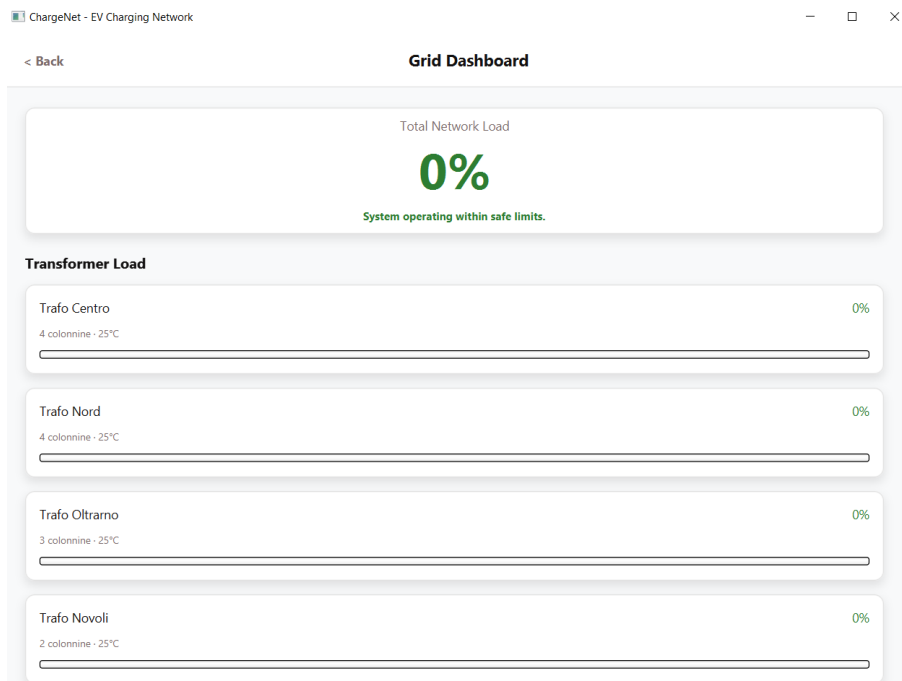


Figura 2.17: Dashboard di affidabilità della rete elettrica.

2.4 Page Navigation Diagram

Il diagramma di navigazione illustra in che modo le diverse schermate dell'applicazione sono collegate tra loro, mostrando il percorso che l'utente compie muovendosi nel sistema in base alle scelte effettuate o ai pulsanti premuti.

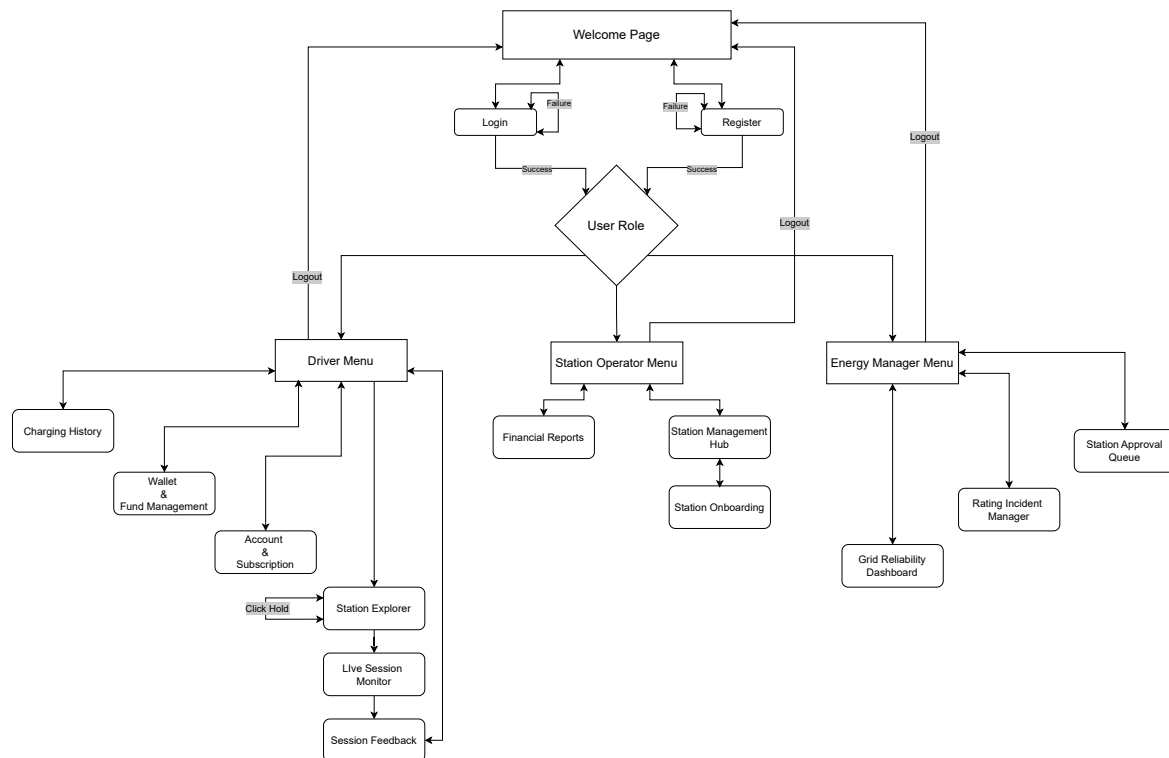


Figura 2.18: Diagramma di navigazione tra le pagine dell'applicazione.

2.5 Package Diagram

Passando all'architettura interna, il codice è organizzato in **quattro livelli sovrapposti** sotto il package base `it.unifi.ing.chargenet`, ciascuno con una responsabilità precisa:

- **presentation** — tutto ciò che l'utente vede: i Controller delle schermate JavaFX e il `NavigationManager`, che decide quale schermata mostrare. Non sa nulla di query o di database: chiede ai servizi di fare qualcosa e ne mostra il risultato.
- **business** — la logica applicativa: i `Service`, le strategie di ricarica e i componenti che fanno girare la simulazione (`GridMonitor`, `GridCluster`, `LoadBalancerService`). È qui che vivono le regole del *cosa deve succedere*.
- **dao** — la traduzione fra oggetti Java e righe di tabella. È l'unico livello che sa com'è fatto il database.
- **domain** — le entità: `ChargingStation`, `ChargingSession`, `Driver`, `PowerTransformer` e le altre. Non sono contenitori di dati passivi, ma oggetti *ricchi*, che sanno fare ciò

che riguarda la loro natura: è la `ChargingStation` stessa a sapere come si calcola il prezzo di una ricarica, ed è la `ChargingSession` stessa a sapere a quali condizioni può essere aperta.

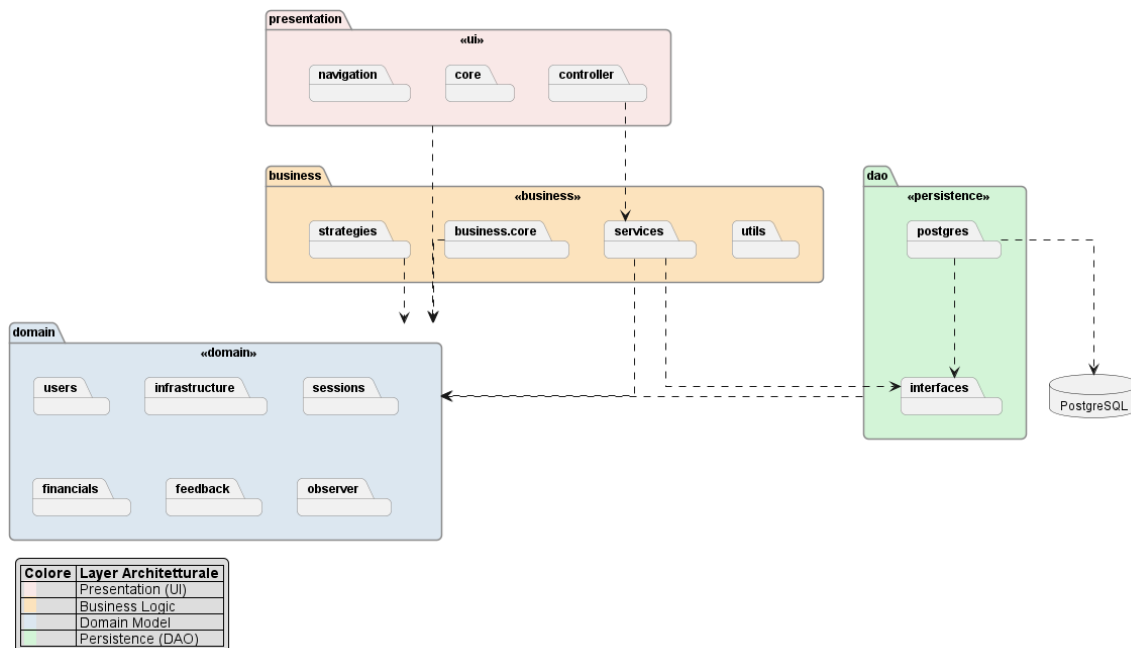


Figura 2.19: Diagramma dei package: verso `domain` entrano frecce da tutti i livelli, ma da esso non ne esce nessuna.

Il dominio non dipende dagli altri livelli

Nella progettazione dell'architettura è stato rispettato il principio per cui nessuna classe del livello `domain` possa dipendere da un altro livello. Le entità non sanno che esistono servizi, controller o database, e nel diagramma questo si può leggere subito: verso il package `domain` entrano frecce da tutti gli altri livelli, ma da esso non ne esce nessuna. Non è solo un'intenzione, ma una proprietà verificabile: nell'intero package `domain` non compare un solo `import` di `business`, `dao` o `presentation`.

Il rispetto di tale principio ha avuto conseguenze concrete. Per esempio il calcolo del prezzo base di una ricarica è stato messo sulla `ChargingStation` (dominio puro) e non sull'interfaccia delle strategie, che si trova nel `business`, proprio perché così sia il dominio che il `business` possono usarlo senza che il dominio debba rivolgersi esternamente. La scelta opposta lo avrebbe reso dipendente da uno strato più esterno e più volatile.

Il beneficio si è visto in modo tangibile nella migrazione da H2 a PostgreSQL, dove entità, servizi e controller sono rimasti intatti perché nessuno di loro sapeva quale database ci fosse sotto: le uniche cose toccate sono state la configurazione della connessione e il bootstrap del database.

2.6 Class Diagram

Il sistema è stato progettato con un'architettura a strati, per mantenere alta coesione e basso accoppiamento (*high cohesion and low coupling*). Nei paragrafi seguenti sono approfondite le classi principali di ciascun livello.

2.6.1 Domain Model

Questo diagramma rappresenta il Domain Model, cioè le entità principali su cui si basa il sistema: utenti, colonnine, trasformatori, sessioni di ricarica e gli oggetti ad esse collegati. Una descrizione più dettagliata sul Domain Model è presente nel paragrafo 3.1

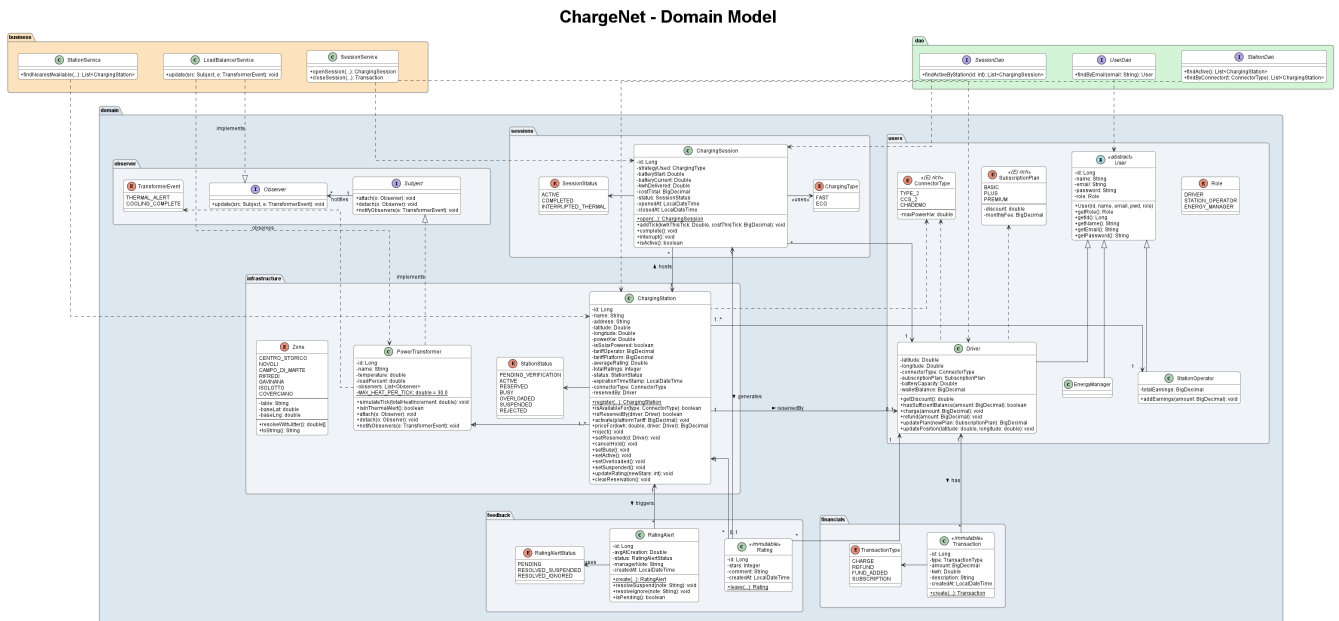


Figura 2.20: Domain Model, diagramma di classe

2.6.2 Business Layer

Il Business Layer è il livello che contiene la logica applicativa del sistema, dove si trovano i servizi che coordinano le entità del Domain Model, le strategie di ricarica e i componenti che gestiscono la simulazione della rete. Il livello è approfondito nel paragrafo 3.2.

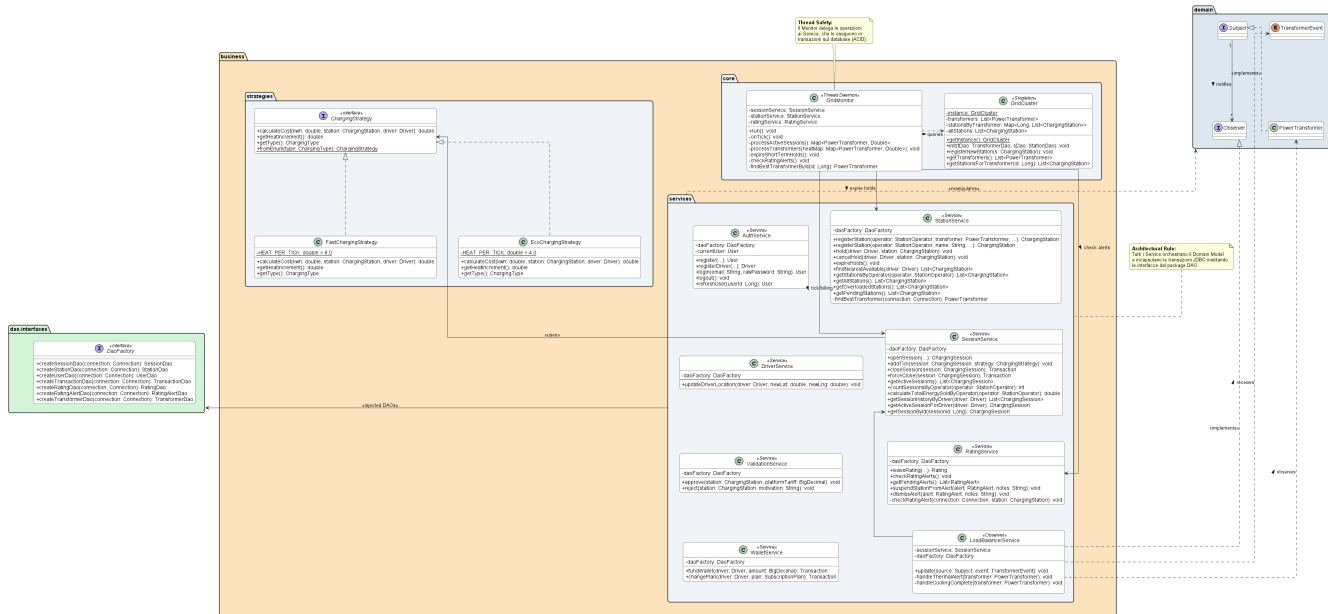


Figura 2.21: Business Layer, diagramma di classe

2.6.3 DAO

Il livello DAO si occupa della comunicazione con il database, traducendo le entità del Domain Model in dati salvabili e recuperabili e garantendone la persistenza.

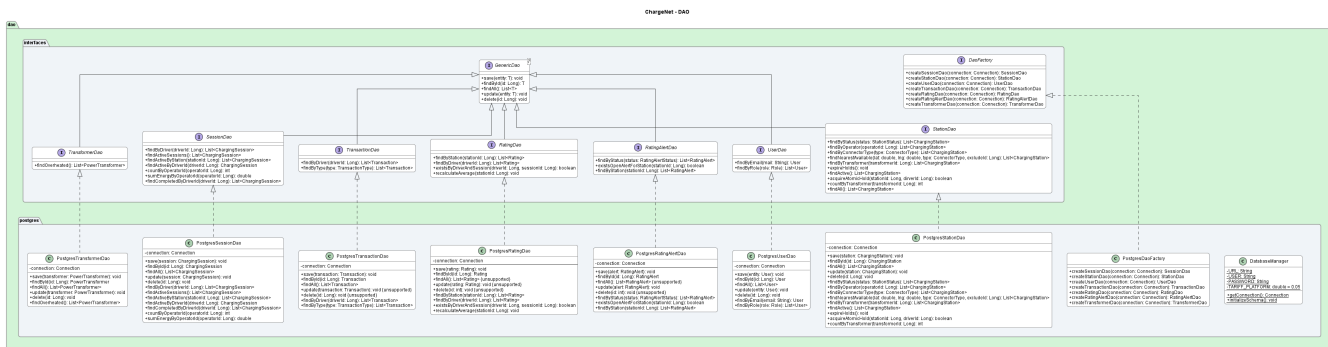


Figura 2.22: DAO, diagramma di classe

2.6.4 Presentation Layer

Il seguente diagramma rappresenta il presentation layer, cioè l'interfaccia grafica dell'applicazione realizzata in JavaFX. Comprende i controller delle varie schermate, che si occupano di mostrare i dati e raccogliere le azioni dell'utente, appoggiandosi ai servizi del livello di business.

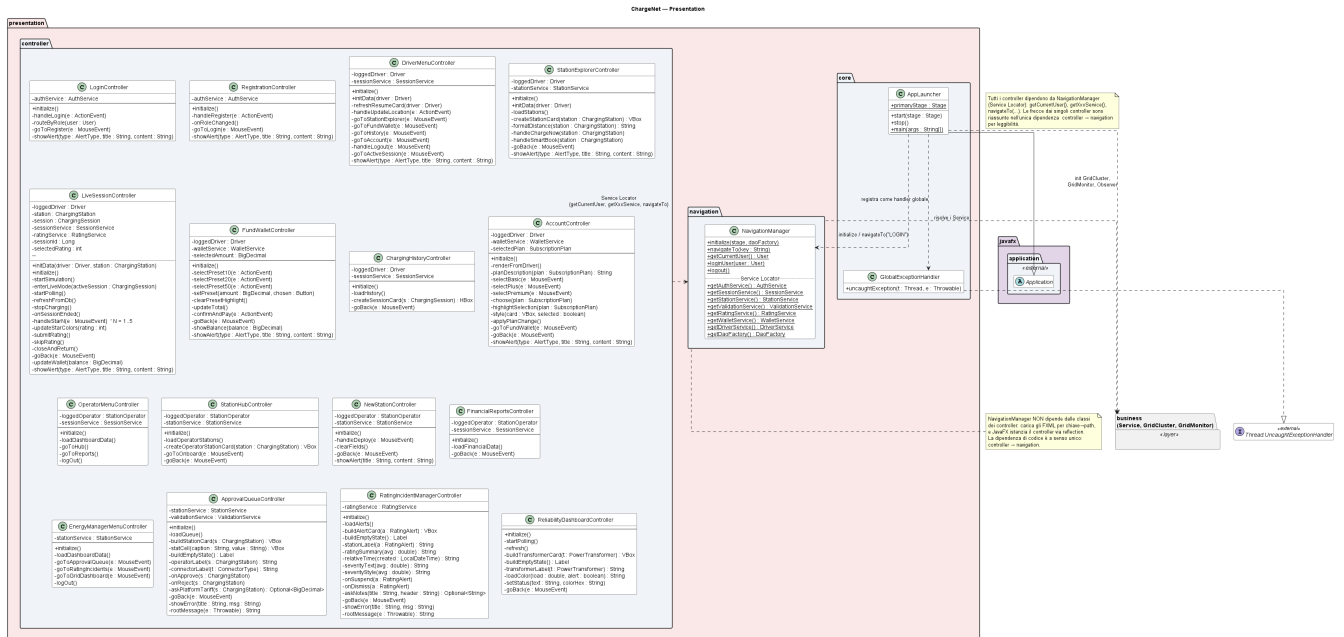


Figura 2.23: Presentation Layer, diagramma di classe

2.7 Sequence Diagram

I diagrammi che seguono descrivono alcuni dei flussi più significativi del sistema, mostrando come le varie classi interagiscono tra loro nel tempo per portare a termine un'operazione.

2.7.1 Ciclo di vita del GridMonitor

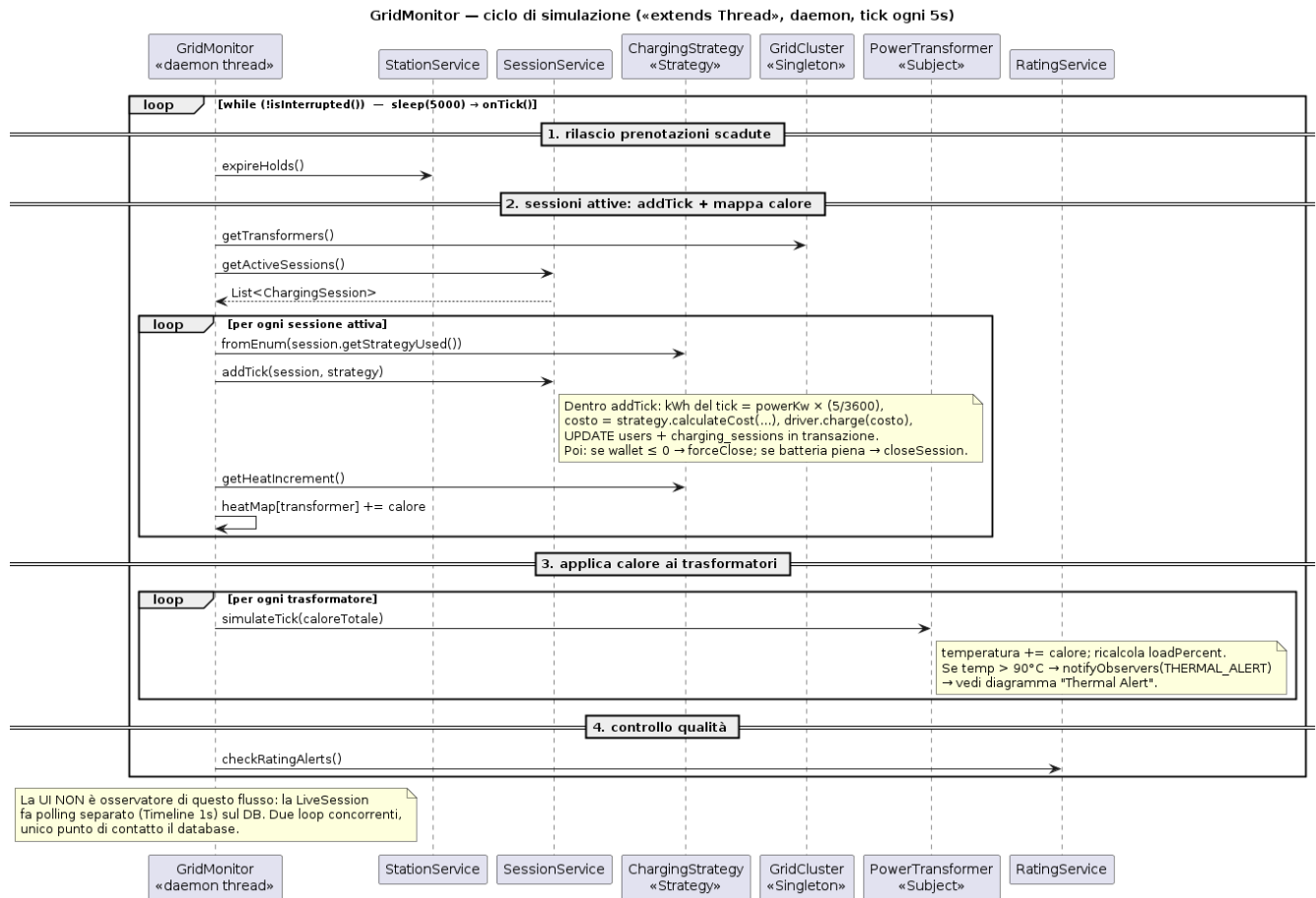


Figura 2.24: Diagramma di sequenza del ciclo di vita del GridMonitor.

Il **GridMonitor** (Fig. 2.24) è il thread daemon che scandisce l'intera simulazione della rete elettrica con un tick fisso di 5 secondi, ed è l'unico componente attivo di propria iniziativa nel sistema: tutti gli altri servizi vengono invocati su richiesta. Ad ogni ciclo il monitor libera prima le prenotazioni scadute tramite **StationService.expireHolds()**, poi recupera da **SessionService** l'elenco delle sessioni attive e, per ciascuna, seleziona la **ChargingStrategy** corrispondente per calcolare kWh erogati e costo del tick, aggiornando in un'unica transazione sia il wallet del Driver sia lo stato della sessione. Lo stesso tick genera un incremento di calore che viene accumulato per trasformatore e applicato a fine ciclo a ciascun **PowerTransformer**: se la temperatura supera i 90°C viene notificato l'evento **THERMAL_ALERT** agli observer registrati, dando origine al flusso descritto in Fig. 2.26. Infine il monitor invoca il controllo di qualità del **RatingService**. La UI non partecipa a questo ciclo come observer: lo stato delle sessioni viene mostrato tramite un polling separato sul database, in questo i due flussi restano disaccoppiati e l'unico punto di sincronizzazione è la persistenza.

2.7.2 Approvazione di una colonnina

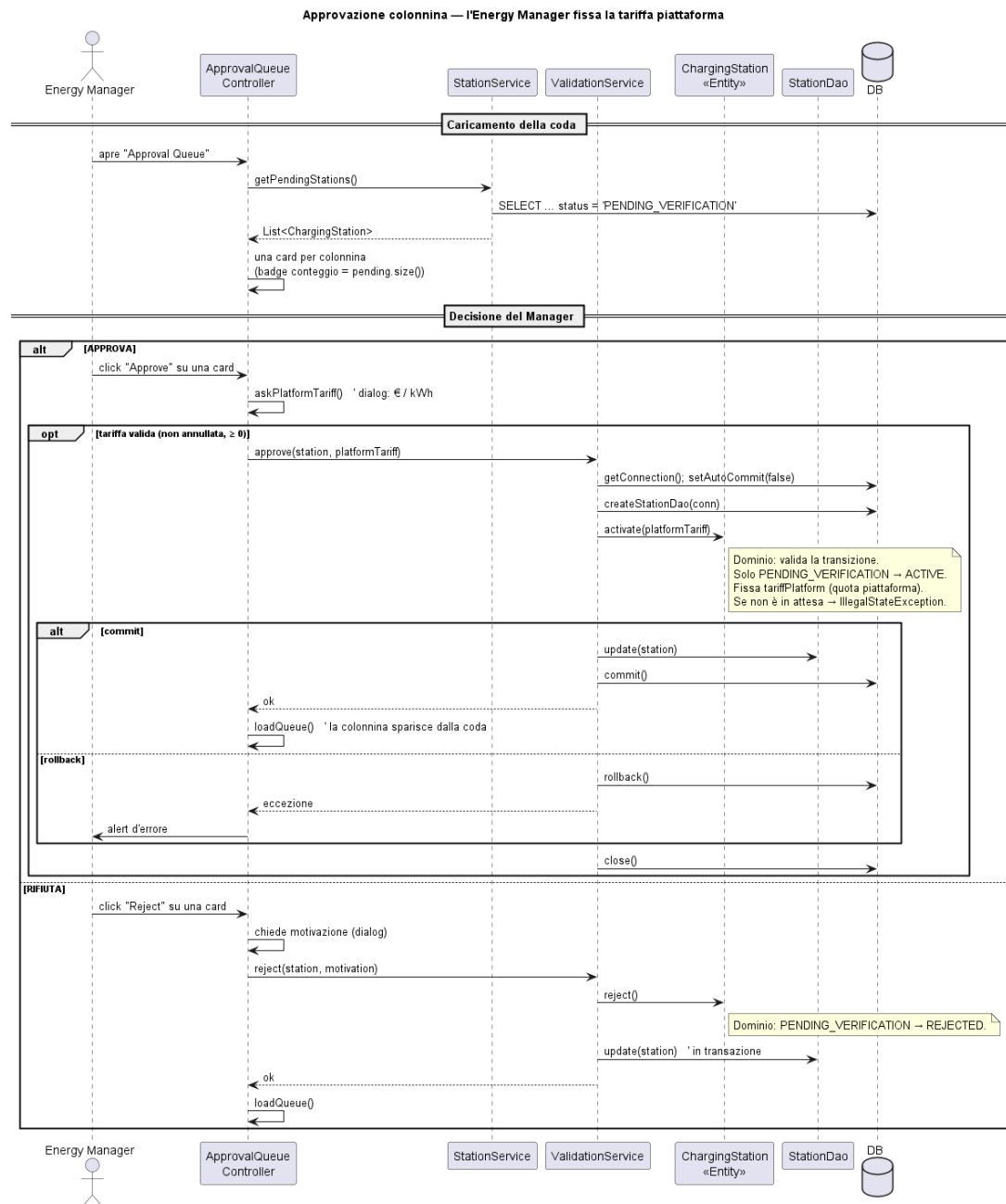


Figura 2.25: Diagramma di sequenza dell'approvazione di una colonnina da parte dell'Energy Manager.

Il flusso di approvazione (Fig. 2.25) regola l'ingresso in esercizio di una nuova colonnina proposta da uno Station Operator. L'Energy Manager apre la coda delle richieste in stato `PENDING_VERIFICATION`, occupata da `StationService.getPendingStations()`, e per ciascuna richiesta può approvare o rifiutare. In caso di approvazione il controller richiede la tariffa di piattaforma da applicare in €/kWh e delega la transizione alla `ValidationService`, che apre una connessione con `setAutoCommit(false)` prima di invocare `activate()` sul dominio: è la `ChargingStation` stessa a validare che la transizione sia legittima solo a partire da `PENDING_VERIFICATION`, sollevando un'eccezione altrimenti,

mentre il servizio si occupa unicamente di gestire la persistenza e il commit o il rollback della transazione in caso di errore. La stessa separazione si ritrova nel percorso di rifiuto, dove il dominio applica la transizione a **REJECTED** dopo che il Manager ha fornito una motivazione. In entrambi i casi la responsabilità di stabilire se un cambio di stato è ammesso resta interamente nel Domain Model, mentre il Business Layer si limita a decidere quando invocarla e a garantirne l'atomicità della persistenza.

2.7.3 Gestione dell'evento di temperatura troppo alta (Thermal Alert)

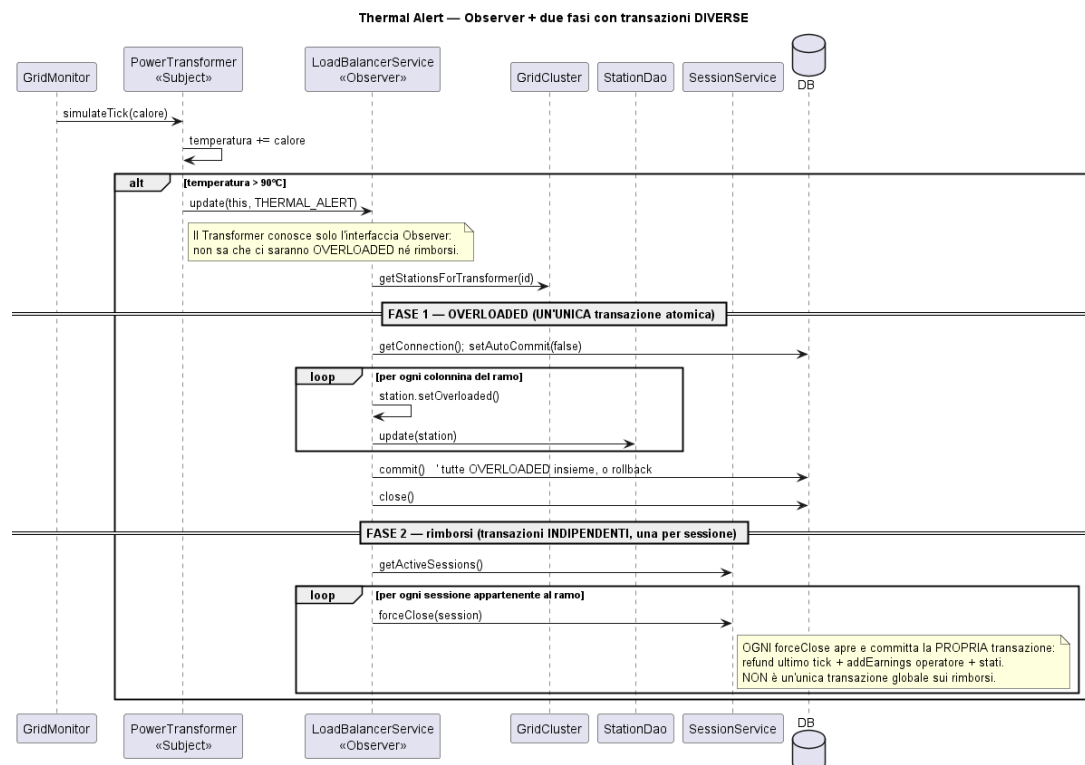


Figura 2.26: Diagramma di sequenza della gestione di una temperatura troppo alta (Thermal Alert).

Il diagramma di Thermal Alert (Fig. 2.26) mostra cosa succede quando un trasformatore si surriscalda. Durante il tick del **GridMonitor**, il **PowerTransformer** riceve il nuovo carico di calore e aggiorna la propria temperatura interna. Se questa supera i 90°C, il trasformatore avvisa il **LoadBalancerService**, che è l'unico componente incaricato di reagire all'emergenza: il trasformatore stesso non sa cosa succederà dopo, si limita a segnalare il problema.

A quel punto il **LoadBalancerService** interviene in due passaggi separati. Nel primo, blocca subito tutte le colonnine collegate a quel trasformatore mettendole in stato **OVERLOADED**, per evitare che qualcuno continui a ricaricare su un ramo elettrico in sovraccarico: questo blocco viene fatto tutto insieme, colonnina per colonnina, come un'unica operazione che riesce o fallisce nel suo complesso. Nel secondo passaggio, il sistema chiude una per una tutte le sessioni di ricarica che erano attive su quelle colonnine, rimborsando ogni Driver coinvolto tramite **SessionService.forceClose()**. Qui però ogni chiusura

viene trattata singolarmente: se il rimborso di una sessione dovesse fallire, le altre sessioni verrebbero comunque chiuse e rimborsate correttamente, e soprattutto le colonnine resterebbero comunque bloccate in sicurezza, indipendentemente da come va il rimborso.

2.8 Database

La gestione dei dati di ChargeNet è affidata a un database relazionale PostgreSQL. In fase di sviluppo si è però lavorato con **H2**, un database che può girare interamente in memoria, che ci è tornato comodo in quanto non richiede installazione e si avvia e si spegne con l'applicazione. Questo ha permesso di scrivere query e provare l'applicazione da subito, rimandando l'installazione e la configurazione del database reale.

Per poter fare poi il passaggio abbiamo dovuto usare H2 fin dall'inizio in modalità `MODE=PostgreSQL`, e scrivere lo schema con tipi e sintassi standard di Postgres (`BIGSERIAL` per le chiavi auto-incrementanti, `DECIMAL`, `TIMESTAMP`, `BOOLEAN`). In questo modo non abbiamo dovuto riscrivere nessuna query dei DAO e nessun Service o Controller si è accorto del cambiamento. E' una dimostrazione concreta dell'indipendenza del dominio descritta nel paragrafo 2.5.

Un'altra operazione necessaria per la migrazione è stata rendere il bootstrap **idempotente**: le istruzioni `DROP TABLE` sono state eliminate in favore di `CREATE TABLE IF NOT EXISTS`, e soprattutto il *seed* dei dati iniziali viene eseguito solo se il database risulta vuoto, con un controllo esplicito che conta le righe della tabella `users`.

Listing 2.1: Bootstrap idempotente: lo schema si crea se manca, il seed solo su DB vuoto

```
// Ri-seedare ad ogni avvio duplicherebbe tutto (e violerebbe
    UNIQUE su email)
if (isDatabaseEmpty(conn)) {
    seedData(conn);
} else {
    System.out.println("[DatabaseManager] DB gia' popolato, seed
        saltato.");
}
```

Un secondo avvio ritrova quindi gli stessi dati invece di duplicarli o azzerarli, che è precisamente la prova che la persistenza funziona.

2.8.1 Schema ER

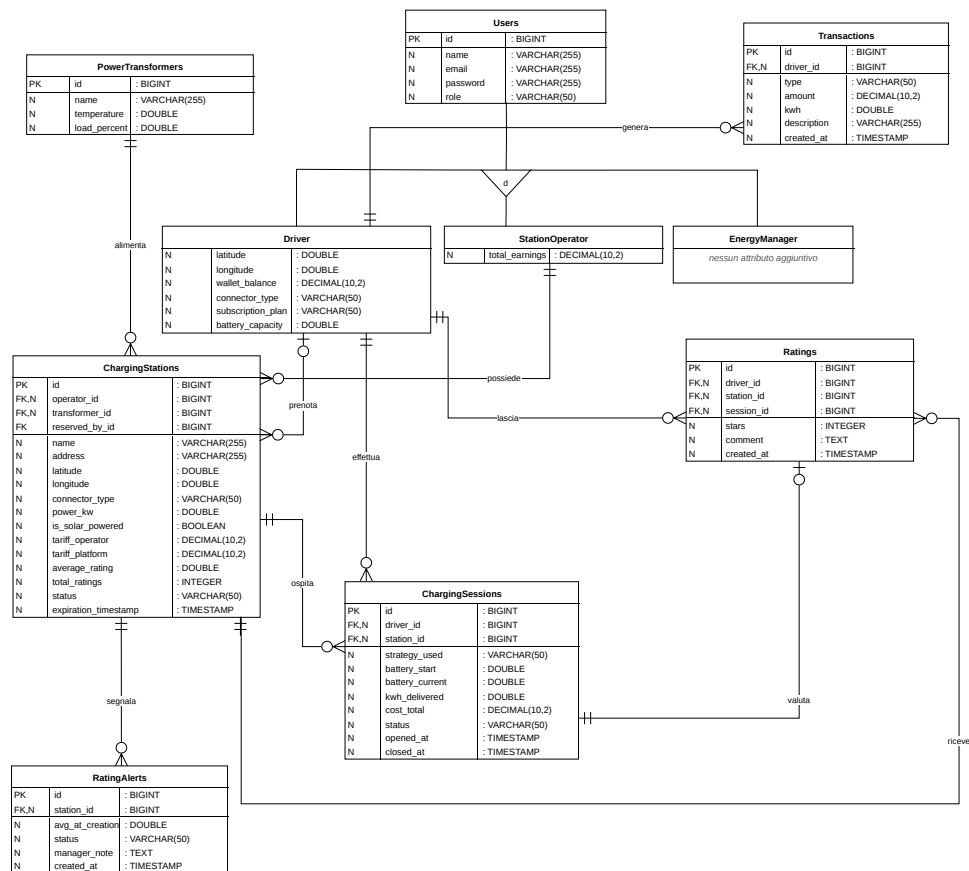


Figura 2.27: Schema Entità-Relazione del database ChargeNet.

Lo schema (Fig. 2.27) mostra la stessa gerarchia già introdotta nel Domain Model: **Users** è un'entità generica che rappresenta ciò che ogni utente ha in comune (identità, credenziali, ruolo), e viene specializzata in tre sottotipi, **Driver**, **StationOperator** ed **EnergyManager**, ciascuno con i propri attributi specifici. Un utente appartiene esattamente a uno dei tre sottotipi, mai a più di uno contemporaneamente, perché la colonna **role** può avere un solo valore.

Le relazioni principali partono dallo specifico sottotipo e non genericamente da **Users**:

- Un **PowerTransformer** alimenta più **ChargingStation**.
- Uno **StationOperator** possiede più **ChargingStation**.
- Un **Driver** può prenotare temporaneamente una colonnina, avviare più **ChargingSession**, generare più **Transaction** e lasciare più **Rating**.
- Una **ChargingStation** ospita più **ChargingSession** e può ricevere più **Rating** o generare più **RatingAlert**.
- Ogni sessione può generare al più una **Rating**.
- **EnergyManager** non compare in nessuna relazione dello schema: il suo ruolo (approvare colonnine, gestire gli alert di qualità) non richiede di possedere o generare entità proprie, ma di agire su entità già esistenti.

2.8.2 Modello Relazionale

La traduzione delle entità semplici segue le regole standard: ogni entità diventa una tabella, ogni relazione (1,n) diventa una chiave esterna sul lato "molti". La generalizzazione è stata invece tradotta con un'unica tabella che raccoglie tutte le colonne di tutti i sottotipi, dove le colonne che non riguardano una riga restano NULL e una colonna discriminatore indica il sottotipo reale.

```
Users(id, name, email, password, role, latitude, longitude, wallet_balance,
      connector_type, subscription_plan, battery_capacity, total_earnings)
```

```
PowerTransformers(id, name, temperature, load_percent)
```

```
ChargingStations(id, operator_id, transformer_id, name, address,
                  latitude, longitude, connector_type, power_kw, is_solar_powered,
                  tariff_operator, tariff_platform, average_rating, total_ratings,
                  status, reserved_by_id*, expiration_timestamp)
```

```
ChargingSessions(id, driver_id, station_id, strategy_used,
                  battery_start, battery_current, kwh_delivered, cost_total,
                  status,
                  opened_at, closed_at*)
```

```
Transactions(id, driver_id, type, amount, kwh, description, created_at)
```

```
Ratings(id, driver_id, station_id, session_id, stars,
        comment, created_at)  UNIQUE(driver_id, session_id)
```

```
RatingAlerts(id, station_id, avg_at_creation, status, manager_note*,
created_at)
```

dove:

```
operator_id, reserved_by_id  si riferisce a  Users(id)
transformer_id  si riferisce a  PowerTransformers(id)
driver_id  si riferisce a  Users(id)
station_id  si riferisce a  ChargingStations(id)
session_id  si riferisce a  ChargingSessions(id)
```

Legenda della notazione

attributo — Chiave Primaria (PK) della tabella.

attributo — Chiave Esterna (FK) verso un'altra tabella.

* — Attributo opzionale (NULL ammesso); tutti gli altri attributi sono obbligatori (NOT NULL).

UNIQUE(...) — Vincolo di unicità su una combinazione di più colonne.

La colonna **role** in **Users** dice se la riga rappresenta un Driver, uno StationOperator o un EnergyManager. Colonne come **wallet_balance** (Driver) o **total_earnings** (StationOperator) esistono per tutte le righe della tabella, ma restano NULL per i sottotipi a cui non appartengono: per esempio non ha senso che **EnergyManager** abbia un valore diverso da null nell'attributo **subscription_plan**. È lo stesso meccanismo descritto nel paragrafo 3.4 a proposito del metodo `extractUserFromResultSet()`, che legge la colonna **role** e ricostruisce l'oggetto Java del sottotipo corretto.

Le due FK verso **Users** in **ChargingStations** (**operator_id** e **reserved_by_id**) modellano invece due relazioni indipendenti — chi possiede la colonnina e chi la sta eventualmente prenotando — ed è per questo che solo la seconda può essere nulla.

Il vincolo **UNIQUE** (**driver_id**, **session_id**) sulla tabella **Ratings** fa sì che anche a livello di database valga la regola che un Driver può lasciare una sola valutazione per ciascuna sessione.

Infine, diversi attributi numerici sono protetti da vincoli **CHECK** direttamente nello schema (es. **stars** tra 1 e 5, **wallet_balance** sempre ≥ 0): questi vincoli sono una seconda difesa rispetto alla validazione già presente nel Domain Model, indipendente da eventuali bug nel codice Java.

È presente un limite, conseguenza della generalizzazione utilizzata: la foreign key **operator_id** garantisce solo che l'id referenziato esista in **Users**, non che quella riga appartenga effettivamente a uno StationOperator; lo stesso vale per **driver_id** rispetto ai Driver. Questo vincolo di coerenza fra ruolo e utilizzo non è esprimibile a livello di schema relazionale standard.

In realtà il problema è comunque risolto dal sistema di tipi di Java, infatti `StationService.register` accetta come parametro uno **StationOperator**, non un generico **User**, e passargli un **Driver** non compilerebbe nemmeno. Quanto detto vale per il *codice* ma non per il database. Una **INSERT** scritta a mano o uno script di popolamento iniziale, possono benissimo scrivere in **operator_id** l'id di un Driver, e il database la accetterebbe subito, perché effettivamente quell'id in **Users** esiste davvero.

Non è solo un'ipotesi teorica, ma è proprio in questo modo che è passato un bug reale del seed, infatti alcune colonnine erano state attribuite per errore al driver invece che all'operatore, alterandoo poi le dashboard dei guadagni. La conclusione corretta è quindi che la coerenza fra ruolo e riferimento è garantita *dal codice*, non *dallo schema*. E' efficace finché tutte le scritture passano dai Service, ma cade non appena qualcosa scrive nel database aggirandoli. Un'alternativa che avrebbe risolto il problema sarebbe stata tradurre la generalizzazione in tabelle separate per sottotipo, che però avrebbe comportato join aggiuntive su ogni login, motivo per cui si è preferita la semplicità della tabella unica, accettando consapevolmente questo limite.

Capitolo 3

Implementazione

Questo capitolo descrive le scelte implementative e l'architettura interna di ChargeNet. Per mantenere la trattazione chiara, non verrà mostrato l'intero codice sorgente, ma ci si concentrerà sull'organizzazione dei componenti principali e sui frammenti di codice che illustrano le logiche più complesse e i pattern utilizzati.

Architettura del Sistema: I livelli principali sono quattro: Domain, Business, Controller e Data Access Object (DAO). L'interazione con il database PostgreSQL è gestita tramite il DAO, che nasconde la complessità delle query SQL al resto dell'applicazione.

3.1 Domain Model

Per avere maggiore ordine, il Domain Model è stato suddiviso in cinque pacchetti principali, ognuno con una responsabilità specifica, raggruppando il codice in base alla finalità e al contesto di utilizzo.

I cinque pacchetti principali sono:

- **Users:** Qui sono definiti gli attori del sistema. Ci sono tre profili principali (*Driver*, *StationOperator* ed *EnergyManager*), che estendono tutti una classe astratta comune per semplificare le operazioni di login e registrazione.
- **Infrastructures:** Rappresenta le parti fisiche della rete. Contiene le colonnine (*ChargingStation*) e i trasformatori (*PowerTransformer*), e serve a gestire parametri come la potenza massima erogabile o lo stato di occupazione.
- **Sessions:** E' la parte che collega l'utente con l'infrastruttura. La classe centrale è *ChargingSession*, che si occupa di far partire la ricarica e di calcolare quanti kWh vengono consumati col passare del tempo.
- **Financials:** Si occupa dei pagamenti e del saldo degli utenti, gestendo le ricevute (*Transaction*) create dopo una ricarica, un rimborso o il pagamento di un abbonamento e facendo in modo che lo storico contabile non possa essere modificato per sbaglio.
- **Feedback:** La parte dedicata alle recensioni. Vengono raccolti i voti (*Rating*) lasciati dai guidatori e fa scattare in automatico un avviso di manutenzione (*RatingAlert*) se la media di una stazione si abbassa troppo.

Nei prossimi paragrafi vedremo più nel dettaglio alcune delle soluzioni implementative che abbiamo adottato in questi cinque moduli.

3.1.1 Users

Questo modulo definisce le identità degli attori tramite un'architettura basata sull'ereditarietà. Dalla classe astratta `User` derivano i tre profili specifici (`Driver`, `StationOperator` ed `EnergyManager`), ognuno con i propri attributi.

Per rispettare il principio dell'incapsulamento, la logica di business è integrata direttamente nelle entità. Un esempio pratico è la gestione economica del `Driver`: quando c'è da scalare del credito, è l'oggetto stesso a fare i controlli sul proprio saldo.

Listing 3.1: Controllo del saldo nel metodo `charge()`

```
public void charge(BigDecimal amount) {
    if (!hasSufficientBalance(amount)) {
        throw new InsufficientBalanceException("Saldo insufficiente");
    }
    this.walletBalance = this.walletBalance.subtract(amount);
}
```

Questo approccio torna utile anche per operazioni più delicate. Il cambio di piano, ad esempio, non addebita il canone intero del nuovo piano, ma solo la differenza rispetto a quello attuale, e blocca i downgrade riusando comunque `charge()` per il controllo del saldo:

Listing 3.2: Aggiornamento del piano tariffario

```
public BigDecimal updatePlan(SubscriptionPlan newPlan) {
    BigDecimal currentFee = (this.subscriptionPlan != null)
        ? this.subscriptionPlan.getMonthlyFee() : BigDecimal.ZERO;
    BigDecimal delta = newPlan.getMonthlyFee().subtract(currentFee)
        .setScale(2, RoundingMode.HALF_UP);

    if (delta.compareTo(BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException(
            "Non è possibile passare a un piano inferiore durante il periodo attivo.");
    }
    if (delta.compareTo(BigDecimal.ZERO) > 0) {
        charge(delta);
    }
    this.subscriptionPlan = newPlan;
    return delta;
}
```

In realtà anche se nel messaggio di eccezione si parla di un downgrade vietato "durante il periodo attivo", nel modello non esiste nessun periodo di abbonamento: non ci sono

né una data di inizio né una scadenza, il Driver ha semplicemente un `SubscriptionPlan` corrente. Il blocco del downgrade è quindi una regola assoluta, ovvero non si scende mai di piano. Non è legata a un vincolo temporale come il messaggio potrebbe lasciare intendere.

Per il collegamento con il database, invece di passare per i costruttori standard, che farebbero scattare validazioni inutili in fase di caricamento, abbiamo dotato ogni entità di un metodo `reconstitute()`, pensato esclusivamente per ripristinare in memoria i dati letti da PostgreSQL.

3.1.2 Infrastructures

L'obiettivo di questo pacchetto è ottenere con il codice i limiti fisici del mondo reale. Per la classe `ChargingStation` era necessario impedire la creazione di oggetti con parametri incoerenti o pericolosi.

Abbiamo quindi nascosto i costruttori, obbligando il resto del sistema a passare per il Factory Method `register()`. In questa fase la stazione esegue quattro controlli in sequenza: coordinate geografiche valide, potenza compresa tra 7 e 350 kW, potenza compatibile col limite fisico del connettore, e tariffa dell'operatore positiva. Basta che uno fallisca per bloccare subito la creazione.

Listing 3.3: Factory Method per la registrazione della stazione

```
public static ChargingStation register(StationOperator operator,
    PowerTransformer transformer,
                                   String name, String address
                                   , Double lat, Double lng
                                   ,
                                   ConnectorType connectorType
                                   , Double powerKw,
                                   Boolean isSolar, BigDecimal
                                   tariff) {

    if (lat < -90.0 || lat > 90.0) {
        throw new InvalidStationParametersException("Latitudine non
            valida");
    }
    if (lng < -180.0 || lng > 180.0) {
        throw new InvalidStationParametersException("Longitudine
            non valida");
    }
    if (powerKw < 7.0 || powerKw > 350.0) {
        throw new InvalidStationParametersException("Potenza non
            valida. Deve essere compresa tra 7 kW e 350 kW.");
    }
    if (powerKw > connectorType.getMaxPowerKw()) {
        throw new InvalidStationParametersException("Errore di
            sicurezza: La potenza richiesta (" + powerKw +
            " kW) supera il limite fisico supportato dal
            connettore " + connectorType.name() +
            " (" + connectorType.getMaxPowerKw() + " kW).");
    }
    if (tariff.compareTo(BigDecimal.ZERO) <= 0) {
        throw new InvalidStationParametersException("La tariffa
            dell'operatore deve essere strettamente positiva.");
    }
    return new ChargingStation(operator, transformer, name, address
        , lat, lng, connectorType, powerKw, isSolar, tariff);
}
```

Nella stessa classe è presente anche il metodo per il calcolo del prezzo di una ricarica, `priceFor(kwh, driver)`. La tariffa è a due componenti: `tariffOperator`, fissata dall'operatore alla registrazione, e `tariffPlatform`, fissata dall'Energy Manager al momento dell'approvazione (paragrafo 3.2.6). Il metodo somma le due quote per ottenere la tariffa al kWh, ma applica lo sconto legato all'abbonamento del Driver solo sulla quota della piattaforma:

Listing 3.4: Calcolo del prezzo: lo sconto tocca solo la quota piattaforma

```
public BigDecimal priceFor(double kwh, Driver driver) {
    BigDecimal discount = (driver != null && driver.
        getSubscriptionPlan() != null)
        ? BigDecimal.valueOf(driver.getSubscriptionPlan().
            getDiscount())
        : BigDecimal.ZERO;

    BigDecimal platformAfterDiscount = tariffPlatform.multiply(
        BigDecimal.ONE.subtract(discount));
    BigDecimal ratePerKwh = tariffOperator.add(
        platformAfterDiscount);

    return ratePerKwh.multiply(BigDecimal.valueOf(kwh))
        .setScale(2, java.math.RoundingMode.HALF_UP);
}
```

In questo modo un piano con sconto riduce ciò che il Driver paga alla piattaforma, ma non tocca il guadagno dell'operatore, che resta invariato qualunque sia il piano del Driver. È ovviamente necessario perché l'abbonamento viene sottoscritto con la piattaforma, non con l'operatore. `priceFor()` è lo stesso metodo richiamato sia da `ChargingSession.open()`, per calcolare il saldo minimo richiesto all'apertura di una sessione (paragrafo successivo), sia dalla `ChargingStrategy` in uso per addebitare ogni tick di ricarica (paragrafo 3.2.4), in questo modo i due punti non possono mai calcolare cifre diverse.

L'altra parte fondamentale del pacchetto è il `PowerTransformer`, che fa da base per il sistema di Load Balancing. Piuttosto che avere un servizio esterno che controlla continuamente i trasformatori, abbiamo reso il trasformatore intelligente applicando il pattern Observer. A ogni ciclo di simulazione, ricalcola la sua temperatura interna; se supera la soglia critica dei 90 gradi, genera autonomamente un `THERMAL_ALERT` per avvisare la rete, senza bisogno di sapere chi o come gestirà effettivamente l'emergenza (vedi paragrafo 3.5.1).

3.1.3 Sessions

Il pacchetto racchiude la logica legata all'erogazione dell'energia. La classe centrale, `ChargingSession`, oltre che a registrare passivamente i dati, come kWh erogati e costi parziali, regola attivamente l'intero ciclo di vita dell'operazione, dalla creazione fino al termine della sessione, che sia andata buon fine o interrotta per surriscaldamento.

Per garantire che una ricarica non inizi in condizioni non valide, l'istanziamento è affidata al metodo factory `open()`. Questo metodo esegue una serie di controlli di dominio: verifica la compatibilità fisica del connettore, controlla che la stazione non sia occupata e delega a `ChargingStation.priceFor()` il calcolo del saldo minimo richiesto per 5 kWh. Questa soglia ovviamente non stima il costo dell'intera sessione, cosa impossibile da prevedere in anticipo perché il Driver può interrompere la ricarica in qualsiasi momento, è solo un valore fisso, scelto per garantire che il Driver abbia credito sufficiente almeno per avviare la ricarica. È lo stesso metodo che userà poi la strategia di ricarica scelta, così il controllo d'ingresso e l'addebito reale non possono mai calcolare cifre diverse:

Listing 3.5: Apertura di una sessione di ricarica

```
public static ChargingSession open(Driver driver, ChargingStation
station, ChargingType strategy, Double batteryStart) {

    boolean isAvailable = station.isAvailableFor(driver.
        getConnectorType()) || station.isReservedBy(driver);
    if (!isAvailable) {
        throw new StationNotAvailableException("La colonnina " +
            station.getName() + " non e' disponibile o non e'
            compatibile col tuo veicolo.");
    }

    BigDecimal minimumRequired = station.priceFor(5.0, driver);

    if (!driver.hasSufficientBalance(minimumRequired)) {
        throw new InsufficientBalanceException("Credito
            insufficiente per avviare la sessione. E' richiesto un
            saldo di almeno " +
                minimumRequired + "€.");
    }

    return new ChargingSession(driver, station, strategy,
        batteryStart);
}
```

Inoltre, la classe incapsula la logica di avanzamento tramite il metodo `addTick()`, che aggiorna in tempo reale i costi e l'energia trasferita, facendo passare automaticamente lo stato della sessione a completato qualora la batteria raggiunga il 100% della sua capacità.

3.1.4 Financials

Qui vengono gestiti i movimenti economici degli utenti, suddividendoli tramite l'enumerazione `TransactionType` (ricariche, rimborsi, abbonamenti e depositi).

Per evitare che l'entità `Transaction` potesse essere modificata a runtime la classe è sprovvista di metodi *setter* e i suoi campi sono valorizzabili unicamente al momento della creazione.

Inoltre la generazione di un nuovo movimento non avviene tramite un normale costruttore, ma è protetta da un metodo *factory* in modo da impedire qualsiasi operazione anomala, come ad esempio i pagamenti a importo zero o privi di causale:

Listing 3.6: Creazione di una transazione contabile

```
public static Transaction create(Driver driver, TransactionType
    type, BigDecimal amount,
                                Double kwh, String description) {

    if (amount == null || amount.compareTo(BigDecimal.ZERO) == 0) {
        throw new IllegalArgumentException("Errore: l'importo non
            puo' essere zero.");
    }
    if (description == null || description.trim().isEmpty()) {
        throw new IllegalArgumentException("Errore: la transazione
            deve avere una descrizione.");
    }
    return new Transaction(driver, type, amount, kwh, description);
}
```

3.1.5 Feedback

A chiudere il Domain Model troviamo la parte dedicata alla qualità del servizio e al supporto. Qui abbiamo deciso di separare l'azione dell'utente (la recensione) dalla conseguente gestione amministrativa (l'allarme per la manutenzione).

Nella classe `Rating`, la creazione di un nuovo feedback è protetta da controlli rigorosi. Abbiamo implementato il metodo factory `leave()`, che non si limita a istanziare la recensione, ma lega quest'ultima alla sessione di ricarica effettiva e impedisce l'inserimento di voti fuori dal range previsto:

Listing 3.7: Creazione di un feedback utente

```
public static Rating leave(Driver driver, ChargingStation station,
    ChargingSession session, Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione: il
            rating deve essere un valore compreso tra 1 e 5 stelle.
            ");
    }
    return new Rating(driver, station, session, stars, comment);
}
```

Una parte interessante di questo modulo si trova nella classe `RatingAlert`. Quando la media di una stazione cala drasticamente, il sistema genera un avviso. Invece di affidare la chiusura di questo ticket a un servizio esterno, abbiamo inserito la logica di transizione di stato direttamente all'interno dell'entità. I metodi per sospendere la colonnina o ignorare l'allarme verificano prima che la segnalazione sia effettivamente ancora in sospeso, evitando che un operatore possa inavvertitamente alterare pratiche già chiuse. La decisione di generare l'alert, invece, non spetta al dominio ma al `RatingService` (vedi paragrafo 3.2.8), che dopo ogni nuova recensione ricalcola la media e verifica la soglia.

Listing 3.8: Chiusura di un alert di manutenzione

```
public void resolveSuspend(String note) {  
    if (isPending()) {  
        this.status = RatingAlertStatus.RESOLVED_SUSPENDED;  
        this.managerNote = note;  
    }  
}
```

3.2 Business Layer

Il Business Layer rappresenta il livello in cui i dati del Domain Model vengono elaborati e trasformati in funzionalità concrete. L'architettura di questo strato è stata progettata seguendo il principio di singola responsabilità (SRP), suddividendo il codice in servizi specializzati (*Services*) e componenti di simulazione (*Core*).

L'implementazione delle diverse logiche di erogazione (*ChargingStrategy*) verrà approfondita nella successiva sezione dedicata ai Design Pattern.

Di seguito vengono analizzate nel dettaglio le classi centrali che governano e dirigono il flusso dell'applicazione.

3.2.1 GridCluster

Il primo problema tecnico che abbiamo affrontato progettando la simulazione è stato il carico sul database. Poiché **GridMonitor** deve ricalcolare consumi e temperature della rete ogni 5 secondi, per ogni sessione attiva e ogni trasformatore, interrogare i DAO ad ogni singolo tick avrebbe significato ripetere le stesse query più volte al minuto per l'intera durata dell'applicazione.

Per risolvere questo, abbiamo implementato **GridCluster**. Si tratta di una cache in memoria, gestita come Singleton, che viene popolata una sola volta all'avvio dell'applicazione. Al suo interno, i dati non sono salvati in semplici liste, ma indicizzati strategicamente tramite mappe Hash per garantire tempi di accesso costanti. Ad esempio, raggruppando le colonnine per ID del trasformatore, il sistema può recuperare l'intero ramo di rete istantaneamente:

```

private Map<Long, List<ChargingStation>> stationsByTransformer;

public void init(TransformerDao tDao, StationDao sDao) {
    this.transformers = tDao.findAll();
    this.allStations = sDao.findAll();

    for (PowerTransformer t : transformers) {
        stationsByTransformer.put(t.getId(), new ArrayList<>());
    }

    for (ChargingStation station : allStations) {
        Long tId = station.getTransformer().getId();
        if (stationsByTransformer.containsKey(tId)) {
            stationsByTransformer.get(tId).add(station);
        }
    }
}

```

3.2.2 GridMonitor

Non avendo a disposizione dei sensori fisici che inviano dati telemetrici, avevamo bisogno di un componente che facesse scorrere il tempo in modo autonomo, senza bloccare l'interfaccia grafica.

Abbiamo quindi esteso la classe `Thread` di Java, impostandola come "Demone" nel costruttore (`this.setDaemon(true);`). In questo modo il ciclo vitale del monitor viene legato a quello dell'interfaccia JavaFX: se l'utente chiude la finestra dell'applicazione, il thread si arresta in modo pulito senza lasciare processi appesi in background.

Il ciclo di esecuzione principale si sveglia ogni 5 secondi e orchestra le operazioni dei vari servizi, aggregando i dati prima di applicarli:

```

private void onTick() {

    expireShortTermHolds();

    Map<PowerTransformer, Double> heatMap = processActiveSessions();

    processTransformers(heatMap);

    checkRatingAlerts();
}

```

In particolare, la fase di elaborazione (`processActiveSessions`) calcola il calore generato dalle singole strategie di ricarica in uso (Fast o Eco) e lo somma all'interno di una *Heat Map*, in modo da notificare i trasformatori una sola volta per ciclo, ottimizzando ulteriormente le prestazioni.

3.2.3 LoadBalancerService

Il `LoadBalancerService` è il componente per cui è necessaria l'esistenza del pattern `Observer` all'interno dell'architettura. Invece di interrogare attivamente la rete per trovare eventuali surriscaldamenti, questo servizio si registra semplicemente in ascolto sui trasformatori.

Quando riceve una notifica di `THERMAL_ALERT`, il servizio reagisce isolando immediatamente il ramo di rete compromesso. L'intervento si divide in due fasi distinte. Nella prima fase viene aperta un'unica transazione che porta a `OVERLOADED` tutte le colonnine collegate al trasformatore, impedendo l'avvio di nuove ricariche. La seconda fase, invece, non è un'unica transazione: ogni sessione viene chiusa e rimborsata singolarmente, con la sua transazione indipendente. È voluto: se il rimborso di un `Driver` fallisse, non deve bloccare anche gli altri, che non hanno colpa di quel fallimento.

Listing 3.9: Gestione dell’alert termico

```
private void handleThermalAlert(PowerTransformer transformer) {

    List<ChargingStation> stations = GridCluster.getInstance().
        getStationsForTransformer(transformer.getId());

    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);
        StationDao stationDao = daoFactory.createStationDao(
            connection);

        for (ChargingStation station : stations) {
            station.setOverloaded();
            stationDao.update(station);
        }
        connection.commit();
    } catch (Exception e) {
        rollbackQuietly(connection);
        System.err.println("Errore nell'aggiornamento delle
            colonnine durante l'alert termico!");
    } finally {
        closeQuietly(connection);
    }

    List<ChargingSession> activeSessions = sessionService.
        getActiveSessions();
    for (ChargingSession session : activeSessions) {
        ChargingStation sessionStation = session.getStation();
        boolean belongsToTransformer = stations.stream()
            .anyMatch(s -> s.getId().equals(sessionStation.
                getId()));
        if (belongsToTransformer) {
            sessionService.forceClose(session);
        }
    }
}
```

Il `catch` mostrato risponde anche a una domanda naturale: cosa succede se la fase 1 fallisce a metà? La transazione va in `rollback`, l'errore viene solo loggato, e l'esecuzione prosegue comunque verso la fase 2 senza alcun controllo sull'esito della prima: le sessioni sul ramo vengono chiuse forzatamente a prescindere da come sia andato l'aggiornamento delle colonnine sul database. È una scelta deliberatamente *fail-safe*: la reazione di emergenza (spegnere le ricariche) non deve mai restare bloccata da un problema di persistenza sullo stato delle colonnine.

Questa seconda fase non è atomica nel suo complesso: ogni chiusura forzata (`forceClose`) apre e chiude una propria transazione indipendente, invece di essere racchiusa in un'unica

transazione che copra tutte le sessioni del ramo. È una scelta voluta, non un limite del sistema: se il fallimento di un singolo rimborso facesse fallire anche gli altri già conclusi con successo, si penalizzerebbero driver che non hanno responsabilità in quel fallimento.

Ogni chiusura forzata produce due effetti economici distinti, salvati nella stessa transazione, il rimborso al Driver e l'accredito allo Station Operator.

Listing 3.10: Rimborso e accredito in forceClose()

```
double kwhLastTick = station.getPowerKw() * (5.0 / 3600.0);
BigDecimal refundAmount = BigDecimal.valueOf(
    strategy.calculateCost(kwhLastTick, station, session.getDriver
        ()));

if (wasActive && station.getOperator() != null) {
    double kwhDelivered = session.getKwhDelivered();
    BigDecimal operatorShare = station.getTariffOperator()
        .multiply(BigDecimal.valueOf(kwhDelivered))
        .setScale(2, RoundingMode.HALF_UP);

    if (operatorShare.compareTo(BigDecimal.ZERO) > 0) {
        User u = userDao.findById(station.getOperator().getId());
        if (u instanceof StationOperator operator) {
            operator.addEarnings(operatorShare);
            userDao.update(operator);
        }
    }
}
```

Il Driver viene rimborsato solo del costo dell'ultimo tick da 5 secondi, cioè l'energia pagata ma non ancora erogata al momento dello stacco. Lo Station Operator, allo stesso tempo, viene comunque accreditato della propria quota sull'intera energia realmente erogata fino a quel punto.

Rientro in servizio delle colonnine

L'emergenza però non è uno stato definitivo, e il `LoadBalancerService` gestisce anche la fase di rientro. Il trasformatore non ha una soglia sola, ma **due**: una di *allarme* a 90 °C, oltre la quale scatta il `THERMAL_ALERT` appena descritto, e una di *rientro* più bassa, a 70 °C. Quando la temperatura, non essendoci più ricariche attive su quel ramo, scende nuovamente sotto i 70 °C, il trasformatore emette un secondo avviso, `COOLING_COMPLETE`, che lo stesso observer intercetta per riportare ad `ACTIVE` tutte e sole le colonnine rimaste in `OVERLOADED`, di nuovo in un'unica transazione.

Listing 3.11: Rientro in servizio dopo il raffreddamento

```
private void handleCoolingComplete(PowerTransformer transformer) {
    List<ChargingStation> stations =
        GridCluster.getInstance().getStationsForTransformer(
            transformer.getId());
    ...
    for (ChargingStation station : stations) {
        if (station.getStatus() == StationStatus.OVERLOADED) {
            station.setActive();
            stationDao.update(station);
        }
    }
    connection.commit();
    ...
}
```

La distanza tra le due soglie è fondamentale perchè evita che il sistema oscilli. Infatti se ci fosse una soglia sola, un trasformatore fermo esattamente sul valore critico riattiverebbe le colonnine, che ricomincerebbero subito a scaldare, facendolo risalire sopra la soglia e provocando un nuovo blocco. Si avrebbe sostanzialmente un ciclo di accensioni e spegnimenti a raffica. Imponendo che per rientrare si debba scendere *ben sotto* il punto in cui si è usciti, si ottiene un margine di raffreddamento reale prima di riaprire il ramo. Il filtro `if (station.getStatus() == OVERLOADED)` garantisce inoltre che il rientro riguardi solo le colonnine spente dall'emergenza, senza risvegliare quelle che si trovano in altri stati per ragioni indipendenti (per esempio sospese dall'Energy Manager per qualità scadente).

3.2.4 SessionService

Mentre il `LoadBalancerService` gestisce le emergenze, il `SessionService` governa l'operatività economica ed energetica: è il servizio più sollecitato del sistema, invocato ad ogni tick del `GridMonitor` per ciascuna sessione in corso.

Addebito progressivo del wallet durante la ricarica

il portafoglio del Driver non viene addebitato al termine della ricarica, ma progressivamente, cinque secondi alla volta, dal `GridMonitor`. Ogni tick calcola l'energia erogata nell'intervallo (la potenza della colonnina moltiplicata per la frazione d'ora corrispondente), ne chiede il costo alla `ChargingStrategy` in uso, e scala subito quella cifra dal saldo:

Listing 3.12: Avanzamento di una ricarica: addebito e persistenza in un'unica transazione

```
public void addTick(ChargingSession session, ChargingStrategy
strategy) {
    double oreTick = 5.0 / 3600.0;
    double kwhThisTick = session.getStation().getPowerKw() *
        oreTick;
    BigDecimal costThisTick = BigDecimal.valueOf(
        strategy.calculateCost(kwhThisTick, session.getStation(),
            session.getDriver()));

    session.addTick(kwhThisTick, costThisTick);
    Driver driver = session.getDriver();
    driver.charge(costThisTick);

    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);

        UserDao userDao = daoFactory.createUserDao(connection);
        SessionDao sessionDao = daoFactory.createSessionDao(
            connection);

        userDao.update(driver);
        sessionDao.update(session);

        connection.commit();
    } catch (Exception e) {
        rollbackQuietly(connection);
        throw new RuntimeException("Errore durante il salvataggio
            del tick di ricarica", e);
    } finally {
        closeQuietly(connection);
    }

    if (driver.getWalletBalance().compareTo(BigDecimal.ZERO) <= 0)
    {
        forceClose(session);
    } else if (session.getStatus() != SessionStatus.ACTIVE) {
        closeSession(session);
    }
}
```

3.2.5 StationService

Gran parte di questo servizio si occupa di gestire il salvataggio e la validazione delle nuove colonnine, delegando i controlli di sicurezza ai Factory Method del dominio. Approfondiamo due aspetti principali: l'assegnazione automatica del trasformatore in fase di registrazione, e la gestione delle prenotazioni (*Short-Term Hold*).

Assegnazione automatica del trasformatore

Quando un operatore registra una nuova colonnina, non è lui a scegliere a quale trasformatore collegarla: il parametro non compare nemmeno nel metodo che i Controller invocano davvero, `registerStation(operator, name, address, ...)`, che internamente delega la scelta a un metodo privato, `findBestTransformer()`.

Listing 3.13: Assegnazione del trasformatore meno carico in fase di registrazione

```
private PowerTransformer findBestTransformer(Connection connection)
{
    StationDao stationDao = daoFactory.createStationDao(connection)
    ;
    List<PowerTransformer> transformers = GridCluster.getInstance()
        .getTransformers();

    final int MAX_STATIONS_PER_TRANSFORMER = 10;
    PowerTransformer best = null;
    int minStations = Integer.MAX_VALUE;

    for (PowerTransformer t : transformers) {
        int connected = stationDao.countByTransformer(t.getId());
        if (connected < MAX_STATIONS_PER_TRANSFORMER && connected <
            minStations) {
            minStations = connected;
            best = t;
        }
    }

    if (best == null) {
        throw new NoTransformerAvailableException(
            "Tutti i trasformatori hanno raggiunto la capacita'
            massima");
    }
    return best;
}
```

Fra tutti i trasformatori della rete, viene scelto quello con meno colonnine già collegate, purché resti sotto un tetto massimo (`MAX_STATIONS_PER_TRANSFORMER = 10`). Se anche il trasformatore meno carico ha già raggiunto il tetto, significa che l'intera rete è saturata, e la registrazione fallisce con `NoTransformerAvailableException` invece di collegare comunque la colonnina a un ramo già pieno.

Prenotazioni (Short-Term Hold)

Il problema in fase di prenotazione è la concorrenza: due Driver potrebbero tentare di riservare la stessa colonnina nell'esatto momento in cui si libera. La soluzione ingenua, *leggo lo stato, controllo che sia **ACTIVE**, scrivo **RESERVED***, è una classica *race condition*: fra la lettura e la scrittura c'è una finestra in cui un secondo Driver può leggere lo stesso stato **ACTIVE** e prenotare a sua volta, e la colonnina finirebbe assegnata a entrambi.

Il servizio non fa quindi due operazioni separate, ma una sola: delega al DAO (`acquireAtomicHold`) un **aggiornamento condizionale**, in cui il controllo e la scrittura vivono nella stessa istruzione SQL e vengono quindi eseguiti atomicamente dal database.

Listing 3.14: La condizione di guardia vive dentro la UPDATE

```
UPDATE charging_stations
  SET status = 'RESERVED', reserved_by_id = ?,
      expiration_timestamp = ?
 WHERE id = ? AND status = 'ACTIVE'
```

È la clausola `AND status = 'ACTIVE'` a far funzionare il meccanismo: se un altro Driver è arrivato un istante prima, la riga non è più `ACTIVE`, la `UPDATE` non aggiorna alcuna riga e restituisce 0. Il DAO riporta quel 0 al servizio, che lo traduce in una `StationNotAvailableException` per il secondo utente.

In realtà a differenza di quanto si potrebbe intuire non si impedisce al secondo Driver di provarci, ma si fa in modo che il suo tentativo fallisca in modo pulito e riconoscibile.

Inoltre è bene specificare che l'applicazione è stata implementata per un utente alla volta, quindi questo scenario di corsa non può materialmente verificarsi nella demo. La prenotazione è l'unico punto del sistema in cui due utenti competerebbero per la stessa risorsa, ed è stata scritta per un eventuale sistema multiutente.

3.2.6 Sicurezza e Validazione

La responsabilità di sicurezza dell'applicazione è divisa in due aree distinte, gestite da servizi separati:

- **Autenticazione (AuthService):** Il sistema non salva mai le credenziali in chiaro. L'hashing delle password è delegato alla libreria esterna **BCrypt**. Inoltre, il flusso di registrazione varia dinamicamente in base al ruolo richiesto, imponendo l'assegnazione di coordinate GPS iniziali esclusivamente ai Driver.
- **Ammissione in rete (ValidationService):** Nessuna colonnina registrata da un operatore entra in rete automaticamente. Alla registrazione nasce in stato `PENDING_VERIFICATION` e vi resta finché un Energy Manager non la approva o la rifiuta.

Il secondo punto merita una precisazione, perché è facile leggere l'approvazione come un semplice "sì/no" amministrativo, mentre nel nostro modello è anche un *atto economico*: **approvare una colonnina significa fissarne la quota di piattaforma**. Al Manager, al momento dell'approvazione, viene chiesta la tariffa in euro per kWh che la piattaforma tratterrà su ogni ricarica erogata da quella colonnina; da quel momento quel valore entra nel calcolo del prezzo di ogni sessione su quella colonnina. Il gestore della rete, cioè, non si limita ad autorizzare: fissa anche il prezzo che la piattaforma applica su ogni ricarica, mentre l'operatore resta libero di fissare il proprio.

La divisione delle responsabilità all'interno del flusso è la parte interessante. Il servizio *non* decide se la transizione di stato sia lecita: si limita a orchestrare la transazione e a chiedere al dominio di applicarla.

Listing 3.15: Approvazione: il Service orchestra, il dominio decide

```
public void approve(ChargingStation station, BigDecimal
platformTariff) {
    if (platformTariff == null || platformTariff.compareTo(
        BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException("La tariffa della
            piattaforma deve essere un valore valido e non negativo.
            ");
    }
    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);
        StationDao stationDao = daoFactory.createStationDao(
            connection);

        station.activate(platformTariff);
        stationDao.update(station);

        connection.commit();
    } catch (Exception e) {
        rollbackQuietly(connection);
        ...
    } finally {
        closeQuietly(connection);
    }
}
```

La regola *"si approva solo ciò che è in attesa"* non vive quindi nel `ValidationService`, ma dentro `ChargingStation.activate()`: è la colonnina stessa a sapere di poter diventare `ACTIVE` soltanto a partire da `PENDING_VERIFICATION`, e a sollevare un'eccezione se qualcuno provasse ad approvare una colonnina già attiva o già rifiutata. Il servizio si occupa solo di *quando* invocare la transizione e di garantirne la persistenza atomica: se il dominio rifiuta, il `rollback` annulla tutto e nulla arriva al database. La stessa struttura si ritrova nel percorso di rifiuto (`reject`), dove il dominio applica la transizione verso `REJECTED` dopo che il Manager ha fornito una motivazione obbligatoria.

3.2.7 Gestione Utente e Portafoglio

La gestione del profilo utente e del suo portafoglio è affidata a due servizi distinti: il `DriverService`, che si occupa della posizione del Driver, e il `WalletService`, che implementa le funzionalità legate ai pagamenti e ai piani tariffari.

DriverService: aggiornamento della posizione

La posizione del Driver viene simulata dall'utility `GpsSimulator`, che genera una coppia di coordinate casuali all'interno dell'area urbana di Firenze. Quando il Driver preme "aggiorna posizione" dal proprio menu, il Controller chiede una nuova lettura al simulatore e la passa al `DriverService`, che prima aggiorna l'oggetto Driver in memoria con la nuova

posizione e successivamente salva tale modifica sul database, eseguendo questi due passaggi in un' unica operazione:

Listing 3.16: Aggiornamento della posizione:

```
public void updateDriverLocation(Driver driver, double newLat,
double newLng) {
    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);
        UserDao userDao = daoFactory.createUserDao(connection);

        driver.updatePosition(newLat, newLng);
        userDao.update(driver);

        connection.commit();
    } catch (Exception e) {
        rollbackQuietly(connection);
        throw new RuntimeException("Errore aggiornamento posizione:
        " + e.getMessage(), e);
    } finally {
        closeQuietly(connection);
    }
}
```

Nota: due modi di simulare la geolocalizzazione

Nel sistema sono presenti due meccanismi distinti per simulare la posizione geografica. Il `GpsSimulator` appena descritto copre il lato Driver: genera un punto casuale sull'intera area urbana di Firenze, per imitare una lettura GPS reale.

Sul lato delle colonnine il problema è diverso: la posizione di una `ChargingStation` deriva dal quartiere scelto in fase di registrazione (enumerazione `Zone`). Registrando però più colonnine nello stesso quartiere, queste assumerebbero le stesse identiche coordinate di base, che rappresenta un problema se volessimo in futuro realizzare una rappresentazione grafica relativa al posizionamento geografico delle colonnine. Per risolverlo, l'enum espone una piccola funzione di *Jitter*, che applica rispetto al centro esatto della zona uno scostamento casuale di circa 500 metri, generando coordinate leggermente diverse per ogni stazione pur restando nel quartiere di riferimento:

Listing 3.17: Distribuzione geografica delle colonnine con jitter

```
public double[] resolveWithJitter() {

    double jLat = (Math.random() - 0.5) * 0.01;
    double jLng = (Math.random() - 0.5) * 0.01;
    return new double[]{ baseLat + jLat, baseLng + jLng };
}
```

WalletService: ricarica del portafoglio

La ricarica del portafoglio è gestita da `fundWallet()`: dopo aver controllato che l'importo sia strettamente positivo, il metodo accredita la cifra sul saldo del Driver e registra la ricevuta corrispondente, il tutto nella stessa transazione:

Listing 3.18: Ricarica del portafoglio:

```
public Transaction fundWallet(Driver driver, BigDecimal amount) {
    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("L'importo della
            ricarica deve essere valido e maggiore di zero.");
    }
    ...
    driver.refund(amount);

    Transaction transaction = Transaction.create(
        driver, TransactionType.FUND_ADDED, amount, 0.0,
        "Ricarica portafoglio (Accredito di " + amount + "\
            texteuro)");

    userDao.update(driver);
    transactionDao.save(transaction);

    connection.commit();
    return transaction;
}
```

Come ogni movimento economico del sistema, la ricarica non si limita ad alzare un numero: genera anche una `Transaction`, che resta la prova contabile dell'operazione.

WalletService: cambio di abbonamento

Un'altra operazione di questo modulo è il cambio di abbonamento, la cui logica di calcolo è già contenuta in `Driver.updatePlan()`. Il `WalletService`, prima di delegare al dominio, effettua un'unica verifica aggiuntiva sul saldo:

Listing 3.19: Verifica del saldo prima dell'upgrade

```
if (delta.compareTo(BigDecimal.ZERO) > 0
    && driver.getWalletBalance().compareTo(delta) < 0) {
    throw new IllegalStateException(
        "Fondi insufficienti: servono EUR " + delta + " per l'
            upgrade a " + plan.name() + ".");
}
BigDecimal charged = driver.updatePlan(plan);
```

3.2.8 Gestione Feedback e Manutenzione

Il flusso di controllo della qualità del servizio è interamente gestito dal `RatingService`. Il ciclo di vita di un feedback inizia nel momento in cui il Driver conclude una ricarica e decide di valutare l'esperienza.

In questa prima fase, il sistema esegue un duplice controllo. Dapprima verifica a livello database che l'utente non abbia già recensito quella specifica sessione; superato questo filtro, delega la validazione del formato del voto (da 1 a 5 stelle) al dominio.

La parte interessante di questo servizio è la logica che si innesca subito dopo il salvataggio. Non essendoci un controllo umano costante sulle singole colonnine, il servizio ricalcola in tempo reale la media della stazione appena recensita. Se tale parametro scende sotto la soglia critica di 2.0 (con un campione minimo di 5 valutazioni), il sistema interviene autonomamente generando un `RatingAlert`.

Listing 3.20: Ricalcolo della media e alert di manutenzione

```
private void checkRatingAlert(Connection connection,
    ChargingStation station) {

    ratingDao.recalculateAverage(station.getId());
    ChargingStation updatedStation = stationDao.findById(station.
        getId());

    Double currentAvg = updatedStation.getAverageRating();
    Integer totalRatings = updatedStation.getTotalRatings();

    if (currentAvg != null && currentAvg < 2.0 && totalRatings !=
        null && totalRatings >= 5) {
        if (!ratingAlertDao.existsOpenAlertForStation(station.getId()
            ())) {
            RatingAlert alert = RatingAlert.create(updatedStation,
                currentAvg);
            ratingAlertDao.save(alert);
        }
    }
}
```

Una volta ricevuta la segnalazione sulla propria dashboard, sarà discrezione dell'Energy Manager decidere se ignorarla (qualora si trattasse di un falso allarme) o sospendere la stazione difettosa.

3.3 Controllers

Ogni schermata dell'applicazione ha un Controller associato, il cui compito è sempre lo stesso: leggere cosa ha fatto l'utente, per esempio un click o un testo inserito, passare la richiesta al Service giusto del Business Layer, e mostrare il risultato a schermo. Per questo motivo la struttura interna dei Controller è quasi identica tra loro, e di seguito ne presentiamo l'elenco raggruppato per area applicativa, con una breve descrizione del ruolo di ciascuno.

Autenticazione

- **LoginController:** Raccoglie email e password e delega l'autenticazione ad **AuthService**, instradando l'utente verso la dashboard corretta in base al ruolo.
- **RegistrationController:** Raccoglie i dati per un nuovo account, incluso il ruolo scelto (Driver, Station Operator, Energy Manager).

Area Driver

- **DriverMenuController:** Contenitore della dashboard Driver: gestisce il menu e sostituisce solo il contenuto centrale ad ogni navigazione.
- **StationExplorerController:** Mostra le colonnine compatibili nelle vicinanze e permette di prenotarne una o avviare direttamente una sessione.
- **LiveSessionController:** Mostra l'avanzamento in tempo reale di una ricarica in corso. Non calcola nulla e non è un observer del **GridMonitor**: rilegge lo stato della sessione dal database una volta al secondo, tramite una **Timeline** JavaFX indipendente dal tick del monitor.
- **ChargingHistoryController:** Elenca le sessioni passate del Driver non mostrando quelle fallite a causa del thermal alert.
- **FundWalletController:** Gestisce la ricarica del saldo del wallet.
- **AccountController:** Mostra i dati del profilo: piano di abbonamento (modificabile), email e veicolo (in sola lettura).

Area Station Operator

- **OperatorMenuController:** Contenitore della dashboard dell'operatore, da cui si accede alla gestione delle colonnine e ai rendiconti economici.
- **NewStationController:** Modulo di registrazione di una nuova colonnina, con i suoi parametri tecnici.
- **StationHubController:** Elenco dettagliato delle colonnine possedute dall'operatore, con stato operativo di ciascuna (online, in sovraccarico, ecc.) e accesso rapido alla registrazione di una nuova.
- **FinancialReportsController:** Riepilogo economico dell'operatore: guadagno totale, energia venduta e numero di sessioni erogate.

Area Energy Manager

- **EnergyManagerMenuController:** Contenitore della dashboard dell'Energy Manager, da cui si accede alle altre schermate di gestione della rete.
- **ApprovalQueueController:** Coda delle colonnine in attesa di verifica: l'Energy Manager può approvarle o rifiutarle.

- **RatingIncidentManagerController:** Elenco degli alert generati automaticamente quando una colonnina riceve valutazioni troppo basse, con le azioni di sospensione o chiusura dell'alert.
- **ReliabilityDashboardController:** Stato in tempo reale della rete elettrica: carico e temperatura di ogni trasformatore, con evidenza visiva di eventuali allarmi termici.

Service Locator

I Controller non li istanziamo noi ma vengono direttamente costruiti mentre carica il file `.fxml` a cui sono associati. Questo esclude la tecnica che sarebbe la più pulita per fornire a un oggetto le sue dipendenze, cioè passargliele nel costruttore (*dependency injection*): il costruttore non lo chiamiamo noi.

La soluzione adottata è il pattern **Service Locator**. Quando un controller ha bisogno di un servizio non se lo fa passare, ma va a chiederlo a un punto centrale noto, il `NavigationManager`, che espone metodi come `getSessionService()`, `getStationService()` e `getCurrentUser()`. Il recupero avviene nel metodo `initialize()`, che JavaFX invoca subito dopo aver costruito il controller e inserito i campi `@FXML`.

In questo modo si garantisce che un Controller non parli mai direttamente al database ma debba sempre passare da un Service. Un pezzo di interfaccia che scavalca la logica di business per andare dritto a un DAO renderebbe impossibile far rispettare le regole di validazione, calcoli, transizioni di stato che nel DAO non esistono.

Indipendenza della LiveSession dal GridMonitor

La schermata di ricarica in tempo reale (`LiveSessionController`) è quella in cui il rapporto fra interfaccia e simulazione si vede meglio, verrebbe da pensare che la schermata sia un observer del `GridMonitor`, notificata ad ogni tick.

In realtà non è assolutamente così, la schermata non calcola nulla e non riceve alcuna notifica: possiede un proprio ciclo di aggiornamento, una `Timeline` JavaFX che una volta al secondo rilegge dal database lo stato della sessione e aggiorna i numeri a video.

Listing 3.21: Il ciclo di lettura della schermata, indipendente dal tick del monitor

```
// Il GridMonitor fa girare i tick, noi osserviamo
pollTimeline = new Timeline(new KeyFrame(Duration.seconds(1), e ->
    refreshFromDb()));
pollTimeline.setCycleCount(Timeline.INDEFINITE);
pollTimeline.play();
```

Il `GridMonitor` scrive ogni 5 secondi, la schermata legge ogni secondo, e i due cicli non comunicano mai direttamente: l'unico punto di sincronizzazione è il database. In questo modo il vantaggio è che se il Driver chiude la schermata, la ricarica prosegue lo stesso, esattamente come una colonnina reale continua a caricare anche quando non si guarda l'app sul telefono. Per rientrare, il menu del Driver mostra una scorciatoia "Resume" verso la sessione ancora in corso. Se la schermata fosse stata un observer del motore del `GridMonitor`, chiuderla avrebbe significato staccare l'ascoltatore, e sarebbe stato molto più facile scrivere per sbaglio un'interfaccia che chiudendosi fermasse la simulazione.

Il dato sempre aggiornato

Un dettaglio implementativo interessante si trova in `FinancialReportsController`: prima di leggere il guadagno totale dell'operatore, il controller non si fida del valore tenuto in sessione, ma lo rilegge direttamente dal database.

Listing 3.22: Aggiornamento del controller con dati freschi

```
User refreshed = NavigationManager.getAuthService().refreshUser(
    loggedOperator.getId());
if (refreshed instanceof StationOperator op) {
    this.loggedOperator = op;
}
BigDecimal revenue = loggedOperator.getTotalEarnings() != null
    ? loggedOperator.getTotalEarnings() : BigDecimal.ZERO;
```

L'oggetto `User` tenuto in sessione contiene l'informazione dei dati dell'utente al momento del login. Alcuni di quei dati cambiano però mentre l'utente naviga: il saldo del Driver scende ad ogni tick di ricarica, i guadagni dell'operatore salgono ad ogni sessione chiusa. Una schermata che mostrasse il valore in memoria mostrerebbe un numero vecchio. La regola, applicata a tutte le schermate che espongono valori che cambiano, è che l'identità e il ruolo si leggano dalla sessione, mentre qualunque valore soggetto a variazione si rilegge dal Service competente ogni volta che la schermata viene aperta. Così si evita un'intera categoria di bug, cioè quelli in cui l'utente vede numeri non aggiornati.

3.4 Persistenza: il livello DAO e il mapping oggetti/tabelle

Il livello DAO si occupa di tradurre le righe delle tabelle SQL in oggetti Java e viceversa. Il problema che questo livello risolve, il disallineamento fra il modello a oggetti e quello relazionale, è lo stesso che affrontano gli ORM (*Object-Relational Mapper*) come Hibernate, la differenza però è che qui non è stato usato alcun ORM. infatti il mapping è scritto a mano, riga per riga, con query SQL esplicite e `PreparedStatement`. In questo modo abbiamo dovuto vedere esattamente cosa accade fra un oggetto Java e una riga di tabella, invece di delegarlo a una libreria che lo nasconde.

Il `GenericDao<T>` definisce le operazioni base che garantiscono la persistenza dei dati:

Listing 3.23: Interfaccia DAO generica

```
public interface GenericDao<T> {
    void save(T entity);
    T findById(Long id);
    List<T> findAll();
    void update(T entity);
    void delete(Long id);
}
```

Ogni entità del dominio ha poi una classe concreta (es. `PostgresUserDao`) che implementa queste operazioni scrivendo query SQL vere tramite `PreparedStatement`, ed esegue la trasformazione in entrambe le direzioni: dall'oggetto Java verso le colonne del-

la tabella al momento del salvataggio, e dalle colonne del `ResultSet` di nuovo verso un oggetto Java al momento della lettura.

Tabella users

La tabella `users` contiene sia i Driver che gli Station Operator che gli Energy Manager, distinti da una colonna `role`. Il mapping deve quindi decidere quale sottoclasse istanziare:

Listing 3.24: Mapping utente da `ResultSet`

```
private User extractUserFromResultSet(ResultSet rs) throws
SQLException {
    Long id = rs.getLong("id");
    String name = rs.getString("name");
    String dbEmail = rs.getString("email");
    String password = rs.getString("password");
    Role role = Role.valueOf(rs.getString("role"));

    if (role == Role.DRIVER) {
        Double lat = rs.getDouble("latitude"); if (rs.isNull())
            lat = null;
        Double lon = rs.getDouble("longitude"); if (rs.isNull())
            lon = null;
        Double cap = rs.getDouble("battery_capacity"); if (rs.
            wasNull()) cap = null;
        String connTypeStr = rs.getString("connector_type");
        var cType = connTypeStr != null ? ConnectorType.valueOf(
            connTypeStr) : null;
        String subPlanStr = rs.getString("subscription_plan");
        var sPlan = subPlanStr != null ? SubscriptionPlan.valueOf(
            subPlanStr) : null;
        BigDecimal balance = rs.getBigDecimal("wallet_balance");
        if (balance == null) balance = BigDecimal.ZERO;
        return Driver.reconstitute(id, name, dbEmail, password, lat
            , lon, cType, sPlan, cap, balance);

    } else if (role == Role.STATION_OPERATOR) {
        BigDecimal earnings = rs.getBigDecimal("total_earnings");
        if (earnings == null) earnings = BigDecimal.ZERO;
        return StationOperator.reconstitute(id, name, dbEmail,
            password, earnings);

    } else if (role == Role.ENERGY_MANAGER) {
        return EnergyManager.reconstitute(id, name, dbEmail,
            password);
    }
    return null;
}
```

Colonne come `wallet_balance` o `total_earnings` esistono nella tabella ma hanno senso solo per un tipo di utente: per un Energy Manager, ad esempio, restano NULL. Il

DAO legge la colonna `role` e usa quel valore per capire quale oggetto Java ricostruire.

Metodo `reconstitute()`

Il mapping non chiama mai il costruttore del Driver direttamente, ma un metodo chiamato `reconstitute()`. La differenza è importante: quando un Driver viene creato per la prima volta, in fase di registrazione, deve passare da un Factory Method che valida i dati in ingresso, perché in quel momento i dati arrivano da un utente e potrebbero essere sbagliati. Quando invece un Driver viene riletto dal database, i dati sono già stati validati in passato (e protetti da vincoli SQL come i `CHECK` dello schema) — rivalidarli ad ogni lettura sarebbe inutile. `reconstitute()` si limita quindi a ricostruire l'oggetto così com'è, senza ripetere controlli già superati.

Ricostruzione parziale (stub)

A volte il DAO non ha bisogno di ricostruire un oggetto per intero, ma solo di una sua parte. Nel recupero di un `RatingAlert`, ad esempio, non serve caricare l'intera colonnina collegata (tariffe, stato, coordinate...): basta sapere il suo nome, da mostrare nella schermata dell'Energy Manager.

Listing 3.25: Ricostruzione parziale di `ChargingStation`

```
ChargingStation station = ChargingStation.reconstitute(
    stId, null, null, stName, null, null, null, null, null,
    null,
    null, null, null, null, null, null, null
);
return RatingAlert.reconstitute(alertId, station, avgAtCreation,
    status, managerNote, createdAt);
```

Questa `ChargingStation` è un oggetto reale detto stub, di cui vengono valorizzati solo i campi necessari, mentre gli altri restano `null`. In questo modo si evita una query aggiuntiva per recuperare dati che non verrebbero comunque mostrati; se però in futuro venisse ripassato a un metodo che si aspetta una colonnina completa, ad esempio per un aggiornamento, andrebbe prima ricaricato per intero.

I vincoli dello schema

Oltre alla validazione fatta dal codice Java, anche lo schema del database applica i propri controlli, indipendentemente da cosa succede nel Business Layer:

Listing 3.26: Vincoli SQL di integrità

```
wallet_balance DECIMAL(10, 2) CHECK (wallet_balance IS NULL OR
    wallet_balance >= 0),
...
stars INTEGER NOT NULL CHECK (stars >= 1 AND stars <= 5),
...
CONSTRAINT uq_driver_session UNIQUE (driver_id, session_id)
```

Se per un errore di programmazione un service costruisse un saldo negativo o un voto fuori dal range 1-5, sarebbe comunque il database a rifiutare l'inserimento: è un secondo controllo, indipendente da eventuali bug nel codice Java.

3.5 Idiomi e Design Pattern

Per risolvere alcuni problemi specifici e per rendere il codice più aperto a future estensioni, sono stati usati alcuni design pattern.

3.5.1 Observer Pattern

Quando un trasformatore (`PowerTransformer`) si scalda troppo si limita a lanciare un avviso, mentre chi lo riceve e decide cosa fare è `LoadBalancerService`.

Listing 3.27: Interfacce Subject e Observer

```
public interface Subject {
    void attach(Observer observer);
    void detach(Observer observer);
    void notifyObservers(TransformerEvent event);
}

public interface Observer {
    void update(Subject source, TransformerEvent event);
}

public enum TransformerEvent {
    THERMAL_ALERT,
    COOLING_COMPLETE
}
```

Il vantaggio è che i due componenti restano indipendenti: per aggiungere un secondo componente che si limiti a registrare gli allarmi, basterebbe iscriverlo come nuovo observer, senza modificare il codice del trasformatore.

3.5.2 Strategy Pattern

Un Driver può scegliere tra due modalità di ricarica, Fast o Eco, e ciascuna ha un proprio modo di calcolare il costo e il calore prodotto. Invece di usare un `if/else`, che andrebbe modificato ad ogni modalità aggiunta in futuro, si è definita un'unica interfaccia comune che ogni modalità implementa diversamente. In questo modo `SessionService` si interfaccia senza sapere quale modalità di ricarica ci sia dietro.

Listing 3.28: Interfaccia delle strategie di ricarica

```
public interface ChargingStrategy {
    double calculateCost(double kwh, ChargingStation station,
        Driver driver);
    double getHeatIncrement();
    ChargingType getType();
}
```

Le due implementazioni concrete differiscono su entrambi i fronti dell'interfaccia. `FastChargingStrategy` applica la tariffa piena, senza sconti oltre a quelli già calcolati da `ChargingStation.priceFor()`, e genera 8.0 gradi di calore per tick; `EcoChargingStrategy` applica invece un ulteriore 10% di sconto sul prezzo base e genera solo 4.0 gradi per tick. Lo sconto Eco è però a carico della sola piattaforma: allo Station Operator viene comunque accreditata la sua quota piena sull'energia erogata (paragrafo 3.2.4), una proprietà verificata esplicitamente in `ChargingStrategiesTest` (Capitolo 4). Grazie al pattern, una terza modalità di ricarica si aggiungerebbe con una nuova classe, senza toccare una sola riga di `SessionService` o del `GridMonitor`.

3.5.3 Factory Method

Per evitare che si creino oggetti senza rispettare le regole del dominio, la creazione di un oggetto passa sempre da un metodo che prima controlla che tutto sia in regola, e solo dopo costruisce l'oggetto:

Listing 3.29: Factory Method per la creazione del rating

```
public static Rating leave(Driver driver, ChargingStation station,
    ChargingSession session,
    Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione: il
            rating deve essere un
            valore compreso tra 1 e 5 stelle.");
    }
    return new Rating(driver, station, session, stars, comment);
}
```

Lo stesso principio si ritrova quando si apre una sessione di ricarica, quando si registra una nuova colonnina o quando si crea un alert di qualità: in tutti questi casi il controllo e la creazione avvengono insieme, così è impossibile ottenere un oggetto "a metà", che esiste ma è in uno stato che non dovrebbe essere permesso.

Listing 3.30: altro esempio di Factory method, qui è usato per creare alert di rating. A differenza degli esempi precedenti, qui il costruttore non deve validare nulla, i dati arrivano già controllati da chi lo invoca, ma resta comunque privato per impedire di costruire un alert in uno stato diverso da `PENDING`.

```
private RatingAlert(ChargingStation station, Double
    avgAtCreation) {
    this.station = station;
    this.avgAtCreation = avgAtCreation;
    this.status = RatingAlertStatus.PENDING;
    this.managerNote = null;
    this.createdAt = LocalDateTime.now();
}

public static RatingAlert create(ChargingStation station, Double
    avgAtCreation) {
    return new RatingAlert(station, avgAtCreation);
}
```

I due esempi qui mostrati sono scelti per illustrare due varianti dello stesso principio (il primo valida, il secondo no, ma resta comunque privato) più che per la loro complessità. I Factory Method con la logica di controllo più ricca sono in realtà altri due, già incontrati nel Domain Model: `ChargingStation.register()`, che esegue quattro controlli fisici in sequenza, e `ChargingSession.open()`, che verifica compatibilità del connettore, disponibilità della stazione e saldo minimo nella stessa chiamata.

3.5.4 Singleton

Il sistema tiene traccia di quali colonnine sono collegate a quale trasformatore. Questa informazione viene consultata ogni 5 secondi dal componente che simula il passare del tempo `GridMonitor` e ogni volta che un trasformatore si surriscalda dal componente che deve bloccare le colonnine.

Se questa informazione fosse duplicata in due copie diverse, si rischierebbe un disallineamento tra i due componenti. La soluzione è garantire che ne esista una sola copia in tutto il programma, caricata una volta sola all'avvio e poi consultata da tutti:

Listing 3.31: Singleton GridCluster per la cache della rete

```
public class GridCluster {

    private static GridCluster instance;

    private List<PowerTransformer> transformers;
    private Map<Long, List<ChargingStation>> stationsByTransformer;
    private List<ChargingStation> allStations;

    private GridCluster() {
        this.transformers = new ArrayList<>();
        this.stationsByTransformer = new HashMap<>();
        this.allStations = new ArrayList<>();
    }

    public static synchronized GridCluster getInstance() {
        if (instance == null) {
            instance = new GridCluster();
        }
        return instance;
    }

    public void init(TransformerDao tDao, StationDao sDao) {
        this.transformers = tDao.findAll();
        this.allStations = sDao.findAll();
        for (PowerTransformer t : transformers) {
            stationsByTransformer.put(t.getId(), new ArrayList<>());
        }
        for (ChargingStation station : allStations) {
            Long tId = station.getTransformer().getId();
            if (stationsByTransformer.containsKey(tId)) {
                stationsByTransformer.get(tId).add(station);
            }
        }
    }
}
```

La parola `synchronized` su `getInstance()` serve perché più componenti girano in parallelo su thread diversi: senza questa protezione, due richieste arrivate nello stesso istante potrebbero entrambe trovare `instance` ancora nulla e crearne due copie per errore. I dati non vengono letti dal database ad ogni richiesta: `init()` viene chiamato una sola volta all'avvio dell'applicazione, e da quel momento in poi `getStationsForTransformer()` risponde all'istante leggendo dalla mappa già in memoria, senza interrogare più il database.

3.5.5 DAO (Data Access Object)

Il pattern che struttura l'intero livello di persistenza è stato descritto nel paragrafo 3.4, ma merita di comparire anche qui, perché è a tutti gli effetti un design pattern e non un

semplice dettaglio implementativo. Il problema che risolve è il disallineamento fra due mondi che parlano lingue diverse: da una parte oggetti Java con i loro riferimenti (una `ChargingSession` che *contiene* un `Driver` e una `ChargingStation`), dall'altra righe fatte di colonne e numeri. Il DAO è il traduttore, ed è l'*unico* punto del sistema che sa com'è fatto il database.

La separazione fra *interfaccia* (`StationDao`) e *implementazione concreta* (`PostgresStationDao`) non è una formalità: è ciò che ha reso quasi gratuita la migrazione da H2 a PostgreSQL. Il resto del sistema conosce solo l'interfaccia e ignora quale implementazione lavori sotto; per questo nessun Service e nessun Controller si è accorto del cambio di database. La costruzione delle implementazioni concrete è a sua volta isolata in una `DaoFactory`, il cui unico compito è fabbricarle.

3.5.6 Unit of Work

Strettamente legato al DAO, e già visto all'opera nel paragrafo 3.2.4, c'è il modo in cui il progetto tratta le operazioni che toccano più tabelle insieme. Un tick di ricarica scrive sul saldo del Driver e sullo stato della sessione; la chiusura di una sessione scrive su ricevute, colonnina, sessione e guadagni dell'operatore. Sono scritture correlate: eseguirne solo una parte lascerebbe il sistema in uno stato incoerente (denaro scalato senza energia registrata, o viceversa).

Il pattern applicato è lo **Unit of Work**: il Service apre *una sola* connessione, disabilita l'auto-commit, e da quella connessione fabbrica tutti i DAO che gli servono. Un punto su cui è bene essere espliciti, perché è la domanda naturale: i metodi dei DAO **non ricevono la connessione come parametro**; la connessione viene loro iniettata al momento della creazione, tramite la factory (`daoFactory.createXxxDao(conn)`). Condividendo la connessione, più DAO condividono automaticamente la stessa transazione, e alla fine un unico `commit` fissa tutto oppure un `rollback` annulla ogni modifica parziale.

3.5.7 Service Locator

L'ultimo pattern è quello che governa l'accesso ai servizi da parte dei Controller, ed è stato discusso nel paragrafo 3.3. Lo si richiama qui perché la sua adozione non è un ripiego, ma una risposta precisa a un vincolo del framework: essendo JavaFX a istanziare i Controller via reflection durante il caricamento degli `.fxml`, la dependency injection via costruttore non è praticabile. Il `NavigationManager` funge quindi da punto centrale noto presso cui i Controller reperiscono i servizi di cui hanno bisogno.

Il contrasto con il resto del sistema è la parte da tenere a mente: **i Controller usano il Service Locator perché non siamo noi a costruirli; i Service usano la Dependency Injection perché siamo noi a costruirli**, e ricevere le dipendenze dall'esterno è precisamente ciò che li rende sostituibili con dei mock nei test (Capitolo 4).

Capitolo 4

Testing

4.1 Strategia di Verifica

I test sono eseguiti su due livelli:

- **Livello 1 – Business Logic:** service, componenti core (`GridCluster`, `GridMonitor`) e strategy, testati isolando ogni dipendenza esterna con Mockito.
- **Livello 2 — Persistence Layer:** i DAO Postgres, testati utilizzando un database H2 reale in modalità `MODE=PostgreSQL`, per verificare che le query SQL scritte a mano funzionino davvero e non solo che le interazioni tra oggetti siano quelle attese.

Non ci sono test sui Controller né sull'interfaccia grafica. I Controller, per come li abbiamo scritti, non contengono logica di dominio: leggono l'azione dell'utente, chiamano un Service e mostrano il risultato. La logica che può davvero sbagliare i conti sta nei Service e nel dominio, ed è lì che abbiamo concentrato i test.

4.1.1 Tecnologie e Strumenti

- **JUnit 5:** framework di esecuzione, con le annotazioni `@BeforeEach/@AfterEach` per il setup e cleanup di ogni test e `@BeforeAll/@AfterAll` per le risorse costose (come la connessione H2) condivise da un'intera classe.
- **Mockito:** isola la classe sotto test sostituendone le dipendenze con oggetti finiti, i *mock*, che il test programma per rispondere come vuole; l'esito dipende così dalla sola logica della classe in esame. Oltre alle interfacce, è stato utilizzato anche per le **chiamate statiche**, con `Mockito.mockStatic()`: senza di esso `DatabaseManager.getConnection()` aprirebbe una connessione reale al database e `GridCluster.getInstance()` restituirebbe il Singleton condiviso da tutta l'applicazione, legando il test a uno stato globale.
- **H2 in modalità `MODE=PostgreSQL`:** usato esclusivamente nei test dei DAO, con lo schema reale caricato all'avvio — lo stesso schema `schema.sql` usato in produzione. Quindi i test verificano le query sulle tabelle reali, non su una loro approssimazione.

4.2 Livello 1: Business Logic

4.2.1 SessionServiceTest

In questa classe è stato testato il `SessionService`. In particolare ci siamo concentrati sul ciclo di vita di una sessione, verificando in sequenza i metodi `openSession()` e `closeSession()`, compresi i flussi alternativi (come il saldo insufficiente). Il motivo che ci ha portato a scegliere questi test è che il `SessionService` è l'unico componente che muove denaro: il primo test verifica che una sessione senza copertura economica venga rifiutata dal dominio *prima* di toccare il database, il secondo che alla chiusura allo Station Operator sia accreditata esattamente la sua quota (tariffa operatore × kWh erogati).

Listing 4.1: Estratto di codice della classe `SessionServiceTest`

```
1 @Test
2 void testOpenSession_InsufficientFunds_ThrowsException() {
3     Driver poorDriver = testDriver(1L, "Povero", "p@test.com",
4         SubscriptionPlan.BASIC);
5     ChargingStation station = ChargingStation.reconstitute(
6         1L, null, null, "Stazione", null, 43.77, 11.25,
7         ConnectorType.TYPE_2,
8         50.0, false, new BigDecimal("0.40"), new BigDecimal(
9             "0.05"), 0.0, 0,
10         StationStatus.ACTIVE, null, null);
11     ChargingStrategy strategy = ChargingStrategy.fromEnum(
12         ChargingType.FAST);
13
14     InsufficientBalanceException ex = assertThrows(
15         InsufficientBalanceException.class, () ->
16         sessionService.openSession(poorDriver, station,
17             strategy, 20.0)
18     );
19     assertTrue(ex.getMessage().contains("insufficiente"));
20
21     verify(daoFactoryMock, never()).createSessionDao(any());
22 }
23
24 @Test
25 void testCloseSession_CreditsOperator() throws Exception {
26     Driver driver = testDriver(3L, "Test", "t@test.com",
27         SubscriptionPlan.BASIC);
28     driver.refund(new BigDecimal("50.00"));
29
30     StationOperator operatorStub = StationOperator.reconstitute(
31         99L);
32     ChargingStation station = ChargingStation.reconstitute(
33         1L, operatorStub, null, "Stazione", null, 0.0, 0.0,
34         ConnectorType.TYPE_2,
35         50.0, false, new BigDecimal("0.30"), new BigDecimal(
36             "0.10"), 0.0, 0,
37         StationStatus.ACTIVE, null, null);
38 }
```

```

29     ChargingSession session = ChargingSession.open(driver,
30         station, ChargingType.FAST, 20.0);
31     session.addTick(10.0, new BigDecimal("5.00"));
32
33     StationOperator fullOperator = mock(StationOperator.class);
34     doReturn(fullOperator).when(userDaoMock).findById(99L);
35
36     sessionService.closeSession(session);
37
38     ArgumentCaptor<BigDecimal> captor = ArgumentCaptor.forClass
39         (BigDecimal.class);
40     verify(fullOperator, times(1)).addEarnings(captor.capture());
41     assertEquals(0, captor.getValue().compareTo(new BigDecimal(
42         "3.00")));
43
44     verify(userDaoMock, times(1)).update(fullOperator);
45     verify(connectionMock, times(1)).commit();
46 }

```

4.2.2 StationServiceTest

In questa classe è stato testato lo `StationService`, concentrandoci sul metodo `hold()`, cioè la prenotazione temporanea di una colonnina. Abbiamo scelto questi due test perché sono speculari e coprono la situazione più delicata del sistema: due Driver che prenotano la stessa colonnina nello stesso istante. La difesa contro la corsa è un `UPDATE` condizionale sul database, e i test verificano entrambi gli esiti — chi vince ottiene la prenotazione e il `commit`, chi perde riceve un'eccezione, non sporca l'oggetto in memoria e chiude la transazione con un `rollback`.

Listing 4.2: Estratto di codice della classe `StationServiceTest`

```

1  @Test
2  void testHold_Success() throws SQLException {
3      Driver mockDriver = mock(Driver.class);
4      when(mockDriver.getId()).thenReturn(1L);
5
6      ChargingStation mockStation = mock(ChargingStation.class);
7      when(mockStation.getId()).thenReturn(100L);
8
9      // Il database da' il via libera all'update atomico
10     when(stationDaoMock.acquireAtomicHold(100L, 1L)).thenReturn
11         (true);
12
13     stationService.hold(mockDriver, mockStation);
14
15     verify(mockStation, times(1)).setReserved(mockDriver);
16     verify(connectionMock, times(1)).setAutoCommit(false);
17     verify(connectionMock, times(1)).commit();
18 }

```

```

18
19 @Test
20 void testHold_StationNotAvailable() throws SQLException {
21     Driver mockDriver = mock(Driver.class);
22     when(mockDriver.getId()).thenReturn(1L);
23
24     ChargingStation mockStation = mock(ChargingStation.class);
25     when(mockStation.getId()).thenReturn(100L);
26
27     // Qualcun altro ha gia' preso la colonnina: il DAO
28     // restituisce false
29     when(stationDaoMock.acquireAtomicHold(100L, 1L)).thenReturn
30     (false);
31
32     StationNotAvailableException exception = assertThrows(
33         StationNotAvailableException.class, () ->
34         stationService.hold(mockDriver, mockStation));
35
36     verify(mockStation, never()).setReserved(any());
37     verify(connectionMock, never()).commit();
38     verify(connectionMock, times(1)).rollback();
39 }

```

4.2.3 GridMonitorTest

In questa classe è stato testato il `GridMonitor`, il thread *daemon* che ogni 5 secondi fa avanzare la simulazione. È il componente che agisce di propria iniziativa, quindi il test invoca direttamente un singolo ciclo (`onTick()`) e ne verifica i due effetti: le deleghe ai servizi (scadenza delle prenotazioni, avanzamento delle sessioni, controllo di qualità) e l'impatto fisico sul dominio, cioè il calore accumulato sul trasformatore. Per osservare la temperatura il trasformatore è un oggetto **reale**, non un mock.

Listing 4.3: Estratto di codice della classe `GridMonitorTest`

```

1 @Test
2 void testOnTickDelegationsAndHeatSimulation() throws Exception
3 {
4     // Trasformatore REALE (non un mock): solo cosi' possiamo
5     // osservarne la temperatura
6     PowerTransformer spyTransformer = PowerTransformer.
7     reconstitute(1L, "Spy", 20.0, 0.0);
8     when(tDao.findAll()).thenReturn(Collections.singletonList(
9     spyTransformer));
10    when(sDao.findAll()).thenReturn(new ArrayList<>());
11    GridCluster.getInstance().init(tDao, sDao);
12
13    ChargingSession activeSession = ChargingSession.
14    reconstitute(100L, null, mockStation,
15    ChargingType.FAST, 20.0, 20.0, 0.0, BigDecimal.ZERO
16    ,

```



```

11         SessionStatus.ACTIVE, LocalDateTime.now(), null);
12     when(sessionServiceMock.getActiveSessions()).thenReturn(
13         List.of(activeSession));
14
15     // onTick() e' privato: lo invochiamo via reflection per
16     // simulare un ciclo
17     Method onTickMethod = GridMonitor.class.getDeclaredMethod("
18         onTick");
19     onTickMethod.setAccessible(true);
20     onTickMethod.invoke(gridMonitor);
21
22     verify(stationServiceMock, times(1)).expireHolds();
23     verify(ratingServiceMock, times(1)).checkRatingAlerts();
24     verify(sessionServiceMock, times(1)).addTick(eq(
25         activeSession), any());
26
27     assertTrue(spyTransformer.getTemperature() > 20.0,
28         "La temperatura deve essere salita per effetto
29         della sessione attiva");
30 }

```

4.2.4 LoadBalancerServiceTest

In questa classe è stato testato il `LoadBalancerService`, l'*observer* che reagisce agli eventi emessi dai trasformatori. Abbiamo scelto i due test che coprono l'emergenza termica e il ritorno alla normalità: all'evento `THERMAL_ALERT` tutte le colonnine del ramo devono passare in sovraccarico e le sessioni in corso devono essere chiuse; all'evento `COOLING_COMPLETE` le sole colonnine rimaste `OVERLOADED` devono tornare attive. Il test verifica anche che la chiusura della sessione sia *delegata* al `SessionService` e non reimplementata qui.

Listing 4.4: Estratto di codice della classe `LoadBalancerServiceTest`

```

1 @Test
2 void testUpdate_ThermalAlert_ClosesSessionsAndOverloadsStations
3     () throws Exception {
4     PowerTransformer transformerMock = mock(PowerTransformer.
5         class);
6     doReturn(100L).when(transformerMock).getId();
7
8     ChargingStation stationMock = mock(ChargingStation.class);
9     doReturn(10L).when(stationMock).getId();
10
11     ChargingSession sessionMock = mock(ChargingSession.class);
12     doReturn(stationMock).when(sessionMock).getStation();
13
14     doReturn(Collections.singletonList(stationMock))
15         .when(gridClusterMock).getStationsForTransformer
16         (100L);
17     doReturn(Collections.singletonList(sessionMock))
18         .when(sessionServiceMock).getActiveSessions();
19 }

```

```

16         loadBalancerService.update(transformerMock,
17             TransformerEvent.THERMAL_ALERT);
18
19         verify(stationMock, times(1)).setOverloaded();
20         verify(stationDaoMock, times(1)).update(stationMock);
21         verify(connectionMock, times(1)).commit();
22
23         // La chiusura non e' reimplementata qui: e' delegata al
24         // SessionService
25         verify(sessionServiceMock, times(1)).forceClose(sessionMock);
26     }

```

4.2.5 ChargingStrategiesTest

In questa classe sono state testate le due strategie di ricarica. Non avendo dipendenze esterne, il test non usa mock ma costruisce oggetti di dominio reali. Abbiamo scelto questo test perché verifica la regola tariffaria più facile da sbagliare: lo sconto dell'abbonamento deve essere applicato solo alla quota della piattaforma, mai su quella dell'operatore, e solo dopo si applica l'ulteriore sconto del 10% in caso venga scelta la modalità Eco.

Listing 4.5: Estratto di codice della classe ChargingStrategiesTest

```

1 private double basePrice(double kwh, SubscriptionPlan plan) {
2     double discount = (plan != null) ? plan.getDiscount() :
3         0.0;
4     BigDecimal platAfter = TARIFF_PLAT.multiply(BigDecimal.ONE.
5         subtract(BigDecimal.valueOf(discount)));
6     BigDecimal rate = TARIFF_OP.add(platAfter);
7     return rate.multiply(BigDecimal.valueOf(kwh)).setScale(2,
8         RoundingMode.HALF_UP).doubleValue();
9 }
10
11 @Test
12 void testEcoCalculateCostWithSubscriptionDiscount() {
13     double kwh = 100.0;
14     Driver premiumDriver = testDriver(3L, "Anna", "anna@test.
15         com", SubscriptionPlan.PREMIUM);
16     double expectedPremium = round2(basePrice(kwh,
17         SubscriptionPlan.PREMIUM) * 0.90);
18
19     assertEquals(expectedPremium, ecoStrategy.calculateCost(kwh,
20         mockStation, premiumDriver), 0.001,
21         "PREMIUM: sconto piano sulla sola quota piattaforma,
22         poi Eco 10%");
23 }

```

4.3 Livello 2: Persistence Layer (DAO)

I test dei DAO non usano mock: le query vengono eseguite davvero su database H2, con tabelle e vincoli reali, creato all'avvio della classe di test con lo stesso file `schema.sql` della produzione.

Nonostante il prodotto finale utilizzi PostgreSQL è un compromesso accettato consapevolmente in quanto H2 gira in memoria, parte in un istante e non richiede un server acceso per lanciare i test.

4.3.1 PostgresStationDaoTest

In questa classe è stata testata la persistenza delle colonnine. Abbiamo scelto `testAcquireAtomicHold` perché è il test più importante dell'intera classe: chiude il cerchio lasciato aperto da `StationServiceTest`, dove la risposta del DAO era simulata. Qui i due tentativi di prenotazione vengono eseguiti sul database vero, uno dopo l'altro, e il secondo deve fallire perché la colonnina non è più `ACTIVE`: è l'`UPDATE` condizionale stesso a impedire la doppia prenotazione. Il secondo test verifica invece l'`ORDER BY` della ricerca per prossimità, con il filtro sulle sole colonnine attive.

Listing 4.6: Estratto di codice della classe `PostgresStationDaoTest`

```
1 @Test
2 void testAcquireAtomicHold() throws SQLException {
3     // Due Driver (ID 1 e 2) e una colonnina ACTIVE inseriti
4     // nel database reale
5     // ... setup delle tabelle omissso ...
6
7     // Il primo guidatore prenota
8     boolean success = stationDao.acquireAtomicHold(1L, 1L);
9     assertTrue(success, "La prima prenotazione deve avere
10         successo: la stazione e' ACTIVE");
11
12     // Rileggiamo direttamente dal database lo stato effettivo
13     // della riga
14     try (Statement stmt = connection.createStatement();
15          ResultSet rs = stmt.executeQuery(
16              "SELECT status, reserved_by_id,
17               expiration_timestamp FROM charging_stations
18               WHERE id = 1")) {
19         assertTrue(rs.next());
20         assertEquals("RESERVED", rs.getString("status"));
21         assertEquals(1L, rs.getLong("reserved_by_id"));
22         assertNotNull(rs.getTimestamp("expiration_timestamp"));
23     }
24
25     // Il secondo guidatore tenta la stessa colonnina: l'UPDATE
26     // non aggiorna nulla
27     boolean failedAttempt = stationDao.acquireAtomicHold(1L, 2L);
28     assertFalse(failedAttempt, "La seconda prenotazione DEVE
29         fallire: la stazione non e' piu' ACTIVE");
```

```

23 }
24
25 @Test
26 void testFindNearestAvailable() {
27     ChargingStation s1 = createMockStation("Vicino", 10.0,
28         10.0, StationStatus.ACTIVE);
29     ChargingStation s2 = createMockStation("Lontano", 50.0,
30         50.0, StationStatus.ACTIVE);
31     ChargingStation s3 = createMockStation("Vicino ma Rotta",
32         10.1, 10.1, StationStatus.OVERLOADED);
33     stationDao.save(s1);
34     stationDao.save(s2);
35     stationDao.save(s3);
36
37     List<ChargingStation> nearest =
38         stationDao.findNearestAvailable(11.0, 11.0,
39             ConnectorType.TYPE_2, null, null);
40
41     assertEquals(2, nearest.size(), "Deve trovare solo le 2
42         stazioni ACTIVE");
43     assertEquals(s1.getId(), nearest.get(0).getId(), "La prima
44         deve essere la piu' vicina");
45     assertEquals(s2.getId(), nearest.get(1).getId(), "La
46         seconda deve essere la piu' lontana");
47 }

```

4.3.2 PostgresSessionDaoTest

In questa classe è stata testata la persistenza delle sessioni di ricarica. Abbiamo scelto il test sull'aggregazione dell'energia venduta perché è una query che il livello di business non potrebbe verificare da solo: la somma è calcolata dal database e non in Java. Il test ne controlla entrambi i rami, cioè il totale su più sessioni completate e il caso di un operatore senza sessioni, dove è il COALESCE della query a dover garantire 0.0 anziché null.

Listing 4.7: Estratto di codice della classe PostgresSessionDaoTest

```

1 @Test
2 void testSumEnergyByOperatorId() {
3     ChargingSession completed1 = ChargingSession.reconstitute(
4         null, Driver.reconstitute(2L), ChargingStation.
5             reconstitute(1L),
6             ChargingType.FAST, 10.0, 80.0, 50.5, new BigDecimal
7                 ("20.00"), // 50.5 kWh erogati
8             SessionStatus.COMPLETED,
9             LocalDateTime.now().minusHours(3), LocalDateTime.
10                 now().minusHours(2));
11
12     ChargingSession completed2 = ChargingSession.reconstitute(
13         null, Driver.reconstitute(2L), ChargingStation.
14             reconstitute(1L),

```

```

11         ChargingType.EC0, 50.0, 100.0, 20.0, new BigDecimal
12             ("8.00"), // 20.0 kWh erogati
13         SessionStatus.COMPLETED,
14         LocalDateTime.now().minusHours(6), LocalDateTime.
15             now().minusHours(4));
16
17     sessionDao.save(completed1);
18     sessionDao.save(completed2);
19
20     double totalEnergy = sessionDao.sumEnergyByOperatorId(1L);
21     double emptyEnergy = sessionDao.sumEnergyByOperatorId(99L);
22
23     assertEquals(70.5, totalEnergy, 0.001, "La somma deve
24         essere 50.5 + 20.0 = 70.5 kWh");
25     assertEquals(0.0, emptyEnergy, 0.001, "Senza sessioni il
26         COALESCE deve garantire 0.0");
27 }

```